



VELLORE INSTITUTE OF TECHNOLOGY (VIT) BHOPAL UNIVERSITY

Bhopal-Indore Highway, Kothrikalan, Sehore, Madhya Pradesh – 466114, India

SCHOOL OF COMPUTING SCIENCE AND ENGINEERING (SCSE)

VisionLab: Intelligent Image & Video Analysis Platform

A Full-Stack Educational Computer Vision Laboratory

Academic Project Evaluation Report

Course Code: **CSE3010** | Course Title: **Computer Vision**

Course Type: Embedded Theory & Lab (LP) | Credits: 3

Student Author:	Poorna Gupta
Registration / Roll No.:	24BAI10091
Academic Programme:	B.Tech. Computer Science & Engineering (Specialization in Artificial Intelligence)
Academic Year:	2026 – 2027
Date of Submission:	September 15, 2026

Submitted in partial fulfillment of the continuous evaluation requirements for the course
CSE3010: Computer Vision.

Contents

2	Introduction	1
2.1	Background of Computer Vision	1
2.2	Academic Coursework Context: CSE3010	1
2.3	Project Title & Motivation: VisionLab	2
2.4	Organization of the Report	2
3	Problem Statement	4
3.1	The Pedagogical Gap in Computer Vision Education	4
3.2	Project Objectives	4
4	Functional Requirements	6
4.0.1	Mode A: Image Analysis	6
4.0.2	Mode B: Video Analysis	7
5	Non-functional Requirements	7
6	System Architecture	9
6.1	Architectural Pattern Overview	9
6.2	Detailed Tier Breakdown	9
6.2.1	Tier 1: Client Presentation & Interaction Layer	9
6.2.2	Tier 2: API Gateway & Network Protocol Layer	9
6.2.3	Tier 3: Classical Computer Vision Core	9
6.2.4	Tier 4: Deep Learning & Video Temporal Engine	11
6.2.5	Tier 5: Telemetry, Export & Benchmarking Layer	11
7	Design Diagrams	12
7.1	Use Case Diagram	12
7.2	Workflow Diagram	13
7.3	Sequence Diagram	14
7.4	Class and Component Diagram	15
7.5	Entity-Relationship (ER) & Data Flow Entity Diagram	17
8	Design Decisions & Rationale	18
8.1	In-Depth Architectural Justifications	18
8.1.1	Why Decoupled Client-Server Over a Monolithic Desktop GUI?	18
8.1.2	Why Stateless REST Over WebSockets for Core Processing?	18
9	Implementation Details	20
9.1	Module 1: Low-Level Image Processing & Filtering (CSE3010 Module 1)	20
9.1.1	Spatial Convolutions and Discrete Filtering	20
9.1.2	Intensity Transformations & Histogram Equalization	21
9.1.3	Optimal Intra-Class Variance Thresholding (Otsu's Method)	21
9.1.4	Mathematical Morphology	21
9.2	Module 2: Feature Extraction & Keypoint Descriptors (CSE3010 Module 3)	21
9.2.1	Multi-Stage Canny Edge Detection	21
9.2.2	Harris Corner Detection	22
9.2.3	Scale-Invariant Feature Transform (SIFT)	22

9.2.4	Histogram of Oriented Gradients (HOG)	23
9.3	Module 3: Unsupervised Image Segmentation (CSE3010 Module 3 & 4) .	23
9.3.1	K-Means Clustering Segmentation	23
9.3.2	Mean Shift Mode-Seeking	24
9.4	Module 4: Deep Learning Object Detection via YOLOv8	24
9.4.1	YOLOv8 Architecture & Anchor-Free Head	24
9.4.2	Bounding Box Regression & Non-Maximum Suppression	24
9.5	Module 5: Multi-Object Tracking via ByteTrack	25
9.5.1	Kalman Filter Kinematic State Prediction	25
9.5.2	Two-Stage Bipartite Association (ByteTrack)	25
9.6	Module 6: Motion Analysis & Optical Flow (CSE3010 Module 4)	25
9.6.1	Optical Flow Constraint Equation	25
9.6.2	Lucas-Kanade Sparse Optical Flow (KLT)	26
9.6.3	Gunnar Farneback Dense Optical Flow	26
9.6.4	Adaptive Background Subtraction (MOG2)	26
9.7	Module 7: Video Keyframe Extraction & Stream Telemetry	26
9.8	Core Code Implementation Snippets	27
10	Screenshots / Results	29
10.1	Mode A: Image Preprocessing and Morphological Transformations	29
10.2	Mode A: Classical Feature Extraction and Geometric Analysis	30
10.3	Mode A: Deep Learning Object Detection via YOLOv8n	31
10.4	Mode B: Video Decoding, Stream Telemetry, and Keyframe Extraction .	32
10.5	Mode B: Multi-Object Tracking with ByteTrack Association	33
10.6	Quantitative Performance Benchmarks	34
11	Testing Approach	35
11.1	Testing Methodology & Strategy	35
11.2	Comprehensive Test Verification Matrix	35
11.3	Automated Integration Test Script	35
11.4	Client-Side Usability and Visual Inspection	37
12	Challenges Faced	38
12.1	Challenge 1: Video Stream Serialization and Memory Consumption	38
12.2	Challenge 2: Cross-Origin Resource Sharing (CORS) and Multipart Boundary Serialization	38
12.3	Challenge 3: Sustaining Real-Time Inference on Commodity CPU Hardware	38
12.4	Challenge 4: Visual Contrast and Ergonomics Across Dynamic 3D Shaders	39
12.5	Challenge 5: Resource Cleanup and Process Leakage in Video Pipelines .	39
13	Learnings & Key Takeaways	40
13.1	Deeper Theoretical Understanding of Computer Vision Foundations	40
13.2	Full-Stack Engineering Competencies	40
13.3	Satisfaction of CSE3010 Student Outcomes	40
14	Future Enhancements	42
14.1	1. Integration of 3D Computer Vision and Stereo Reconstruction	42
14.2	2. WebRTC Live Camera Stream Processing	42
14.3	3. Instance Segmentation & Polygonal Masking	42

14.4	4. Client-Side Edge Inference via WebAssembly (Wasm)	42
14.5	5. Custom Convolution Kernel Playground	43
15	References	44

List of Figures

1	High-Level 5-Tier System Architecture Diagram of VisionLab.	10
2	Use Case Diagram of the VisionLab Academic Platform.	12
3	End-to-End Processing Workflow Diagram of VisionLab.	13
4	Sequence Diagram of Request Ingestion, Computation, and Response Rendering.	14
5	Component and Class Dependency Architecture of VisionLab.	16
6	Entity-Relationship (ER) & Data Flow Diagram.	17
7	VisionLab Mode A: Low-Level Image Preprocessing, Filtering, and Morphological Analysis Interface.	29
8	VisionLab Mode A: Canny Edge Detection and Geometric Feature Extraction with Telemetry Analytics.	30
9	VisionLab Mode A: Real-Time Deep Learning Object Detection using YOLOv8n with Interactive NMS Sliders.	31
10	VisionLab Mode B: Video Stream Decoding, Telemetry Extraction, and Temporal Keyframe Strip.	32
11	VisionLab Mode B: ByteTrack Multi-Object Tracking with Trajectory Histories and Class Statistics.	33

List of Tables

1	Key Architectural Design Decisions and Rationale.	19
2	Empirical Execution Latency and Computational Profiling across VisionLab Modules.	34
3	VisionLab Comprehensive Test Verification Matrix.	36

2 Introduction

2.1 Background of Computer Vision

Computer Vision (CV) is an interdisciplinary branch of artificial intelligence and computational science that seeks to emulate human visual perception through algorithmic analysis of digital images and video sequences. Unlike simple image storage or biological optical reception, computer vision focuses on transforming raw photon intensity matrices $I(x, y)$ into high-level semantic understanding, spatial structure, and temporal dynamics.

Historically, computer vision evolved from classical image processing techniques—characterized by 2D spatial filtering, frequency-domain transformations, and mathematical morphology—into mid-level structural representations such as scale-space interest points, edge boundaries, and gradient histograms. In recent years, the discipline has been revolutionized by deep convolutional neural networks (CNNs) and transformer architectures, enabling real-time object classification, boundary-box localization, and persistent multi-object tracking in dynamic scenes.

2.2 Academic Coursework Context: CSE3010

This academic project is conceptualized, designed, and engineered in strict alignment with the curriculum of **CSE3010: Computer Vision**, an advanced course offered at **Vellore Institute of Technology (VIT) Bhopal University**. As detailed in the official course syllabus (Course Code: CSE3010, Course Type: LP, Credits: 3), the course aims to develop deep theoretical rigor and hands-on engineering capabilities across five foundational modules:

1. **Module 1 — Digital Image Formation and Low-Level Processing:** Image formation models, orthogonal and projective transformations, spatial convolutions, discrete filtering, noise suppression, histogram processing, and intensity enhancement.
2. **Module 2 — Depth Estimation and Multi-Camera Views:** Epipolar geometry, homography matrices, fundamental and essential tensors, binocular stereopsis, RANSAC, and 3D reconstruction principles.
3. **Module 3 — Feature Extraction and Image Segmentation:** Multi-stage edge detection (Canny, LoG, DoG), geometric line extraction via Hough Transform, corner interest points (Harris, Hessian Affine), scale-space invariant descriptors (SIFT, SURF, HOG), and segmentation paradigms (K-Means, Mean Shift, Region Growing, and edge-based contour parsing).
4. **Module 4 — Pattern Analysis and Motion Analysis:** Statistical pattern clustering (K-Means, Gaussian Mixture Models), supervised classification (Bayes, KNN, Artificial Neural Networks), motion dynamics (optical flow equations, Lucas-Kanade KLT feature tracking, Gunnar Farneback polynomial expansion), and adaptive background subtraction (MOG2, KNN).
5. **Module 5 — Shape From X & Advanced Contemporary Vision:** Surface photometric reflection models (Phong reflection, albedo recovery), shape from texture, color, and motion cues, as well as state-of-the-art deep learning detectors.

Furthermore, the project satisfies the **Student Outcomes (SO: a, b, c, l)** mandated by the university:

- **SO-a:** Ability to apply mathematical, scientific, and computing principles to discrete image domains.
- **SO-b:** Ability to analyze real-world vision problems and define computational requirements.
- **SO-c:** Ability to design, implement, and evaluate an integrated end-to-end computer vision software system.
- **SO-l:** Ability to apply algorithmic principles and computer science theory in the modeling and design of robust computer-based systems.

2.3 Project Title & Motivation: VisionLab

To bridge the persistent gap between mathematical equations taught in lecture halls and practical software engineering, this project introduces:

“VisionLab — Intelligent Image & Video Analysis Platform”

VisionLab is a full-stack, browser-based, high-performance Computer Vision laboratory. It encapsulates over 30 fundamental algorithms spanning classical spatial filtering, feature geometry, unsupervised segmentation, deep learning detection, and video motion dynamics. Rather than serving as a black-box wrapper around external cloud APIs, VisionLab implements genuine Computer Vision algorithms utilizing **OpenCV 4.10**, **NumPy 2.0**, **SciPy**, **scikit-image**, and **Ultralytics PyTorch YOLOv8**.

The primary motivation is pedagogical accessibility: allowing students, faculty, and evaluators to interactively adjust algorithmic parameters in real time (e.g., standard deviation σ , gradient hysteresis thresholds, cluster counts k , corner sensitivity coefficients, and temporal decimation factors) and immediately observe comparative split-screen visual transformations and quantitative telemetry dashboards.

2.4 Organization of the Report

The remainder of this report is structured as follows:

- **Section 2** defines the core *Problem Statement* and educational challenges addressed.
- **Section 3** articulates the *Functional and Non-Functional Requirements*.
- **Section 4** details the multi-tiered *System Architecture*.
- **Section 5** illustrates the structural *Design Diagrams* (Use Case, Workflow, Sequence, Class/Component, and Entity-Relationship).
- **Section 6** explains the technical *Design Decisions and Rationale*.
- **Section 7** elaborates on the mathematical and programmatic *Implementation Details* for all 7 modules.

- **Section 8** presents the *Screenshots, Results, and Quantitative Discussion*.
- **Section 9** discusses the systematic *Testing Approach and Verification Matrix*.
- **Section 10** reflects on *Challenges Faced and Key Takeaways*.
- **Section 11** outlines *Future Enhancements* and compiles formal *Academic References*.

3 Problem Statement

3.1 The Pedagogical Gap in Computer Vision Education

Teaching and learning Computer Vision at the undergraduate engineering level presents unique pedagogical hurdles. Unlike standard software development, where program correctness can often be evaluated via Boolean unit tests or terminal printouts, computer vision algorithms operate on continuous spatial matrices and stochastic image noise. Consequently, comprehension depends heavily on empirical visual validation.

In conventional university environments, including the practical sessions of CSE3010, students encounter the following systemic problems:

1. **Isolation of Code and Mathematical Intuition:** Students typically execute static Python scripts from a command-line interface or run fragmented cells within Jupyter notebooks. When tuning algorithmic parameters—such as the standard deviation σ of a Gaussian smoothing kernel, the upper/lower hysteresis limits of the Canny edge detector, or the sensitivity factor k of the Harris corner detector—students must repeatedly edit code, recompile, save output raster files, and open image viewers manually. This high-friction, asynchronous iteration cycle obstructs intuitive understanding of parameter sensitivity.
2. **The Classical vs. Modern Dichotomy:** Curricula typically bifurcate computer vision into two disconnected eras: classical handcrafted feature engineering (Sobel, Harris, SIFT, HOG) and modern deep convolutional neural networks (YOLO, ResNet). Students frequently fail to comprehend that modern deep learning models still rely on spatial convolutions, gradient representations, and non-maximum suppression (NMS) principles derived from classical operators. A platform demonstrating both domains within a unified paradigm is missing.
3. **Invisibility of Temporal Dynamics in Video Processing:** Core video analysis concepts—such as the brightness constancy constraint in optical flow:

$$I_x u + I_y v + I_t = 0 \quad (1)$$

and multi-target trajectory association using Kalman filters and the Hungarian algorithm—are inherently continuous and dynamic. Static figures in textbooks and lab manuals fail to convey how motion vector orientations respond to camera pan versus object translation, or how object identifiers are maintained during occlusion.

4. **Absence of Real-Time Comparative Visualization:** Standard evaluation environments lack simultaneous split-screen rendering. Comparing an unenhanced histogram against its equalized counterpart, or comparing raw edge gradients against non-maximum-suppressed contours, requires side-by-side juxtaposition that simple scripts do not provide.

3.2 Project Objectives

To address these fundamental issues, **VisionLab** was engineered to achieve the following specific objectives:

- **Objective 1 (Comprehensive Implementation):** Implement an extensive, authentic suite of over 30 computer vision algorithms covering low-level filtering, feature extraction, segmentation, deep learning detection, and video motion dynamics without resorting to simulated approximations or third-party cloud APIs.
- **Objective 2 (Interactive Hyperparameter Sweeps):** Provide real-time UI controls (sliders, numerical counters, radio matrices) enabling live mathematical adjustment of convolution kernels (3×3 to 31×31), hysteresis thresholds (5 to 400), cluster counts ($k = 2$ to 10), and temporal decimation steps (1 to 30).
- **Objective 3 (Synchronized Dual-Pane Visualization):** Render original and processed media side-by-side with synchronized zoom and coordinate tracking to facilitate immediate visual delta perception.
- **Objective 4 (Integrated Analytical Telemetry):** Pair visual renders with quantitative analytical dashboards (intensity histograms, category frequency donuts, polar motion vector charts, and tracking telemetry tables).
- **Objective 5 (Zero-Friction Portability):** Package the entire system with an automated, zero-configuration local launch mechanism (`start.bat`) that provisions its own virtual environment and serves a lightweight, modern web frontend without requiring heavy JavaScript compilation frameworks.

4 Functional Requirements

The functional capabilities of VisionLab are categorized across the platform's two core operational modes (Mode A: Image Analysis and Mode B: Video Analysis) alongside core platform export and architectural utilities:

4.0.1 Mode A: Image Analysis

- **FR-1.1 (Low-Level Filtering & Spatial Convolutions):** The system shall support image smoothing and enhancement operations including Gaussian Blur (kernel sizes 3×3 to 31×31), Median Filtering (window sizes 3×3 to 21×21), and Laplacian high-pass sharpening via discrete spatial convolution kernels:

$$K_{\text{Laplacian}} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad (2)$$

- **FR-1.2 (Intensity & Histogram Operations):** The system shall generate pixel intensity distribution histograms across grayscale and individual RGB color channels, and perform global Histogram Equalization to enhance low-contrast inputs.
- **FR-1.3 (Thresholding Operations):** The system shall implement global and local adaptive thresholding, including Otsu's optimal intra-class variance minimization, Adaptive Mean thresholding, and Adaptive Gaussian thresholding.
- **FR-1.4 (Mathematical Morphology):** The system shall provide morphological transformations including Dilation, Erosion, Opening, Closing, and Morphological Gradient using Rectangular, Elliptic, and Cross structuring elements.
- **FR-2.1 (Edge & Gradient Detection):** The system shall extract image boundaries using Sobel horizontal/vertical first-order derivatives, Laplacian second-order derivatives, and the multi-stage Canny edge detector with tunable hysteresis thresholds.
- **FR-2.2 (Geometric Feature Detection):** The system shall compute the Probabilistic Hough Line Transform to detect collinear structures within gradient maps.
- **FR-2.3 (Corner & Keypoint Detection):** The system shall compute Harris corner response maps based on the structure tensor M , and extract scale-invariant keypoints using Difference-of-Gaussians (DoG) SIFT feature extraction.
- **FR-2.4 (Dense Feature Descriptors):** The system shall extract Histogram of Oriented Gradients (HOG) feature vectors across normalized spatial cell grids.
- **FR-3.1 (Clustering-Based Segmentation):** The system shall segment images into k color clusters using K-Means vector quantization in RGB/CIELAB space and Mean Shift density mode seeking.
- **FR-3.2 (Region & Contour Segmentation):** The system shall support seeded 4-connected Region Growing and edge-based contour hierarchical polygon extraction.

- **FR-4.1 (Deep Learning Detection):** The system shall perform single-shot object detection using YOLOv8n, predicting bounding boxes, class labels (80 COCO classes), and confidence probabilities.
- **FR-4.2 (Interactive NMS Tuning):** The system shall enable live manipulation of the confidence threshold (0.10 to 0.90) and Intersection-over-Union (IoU) Non-Maximum Suppression threshold (0.10 to 0.90).

4.0.2 Mode B: Video Analysis

- **FR-5.1 (Video Decoding & Stream Telemetry):** The system shall ingest standard video codecs (MP4, AVI, MOV, MKV) and extract resolution, frame rate (FPS), frame count, duration, and file size.
- **FR-5.2 (Keyframe Temporal Sampling):** The system shall extract N evenly-spaced keyframes across the timeline and render a horizontal visual summary strip with frame index markers.
- **FR-6.1 (Multi-Object Tracking):** The system shall integrate YOLOv8n with the ByteTrack algorithm, maintaining persistent tracking IDs across temporal frames via Kalman filter state prediction and Hungarian matching.
- **FR-6.2 (Temporal Decimation):** The system shall allow users to adjust the frame step parameter (1 to 30) to optimize the trade-off between tracking accuracy and processing latency.
- **FR-6.3 (Tracking Telemetry):** The system shall display an annotated keyframe grid, category frequency distribution charts, and a detailed tracking log table.
- **FR-7.1 (Dense Optical Flow):** The system shall compute Gunnar Farneback dense optical flow fields and visualize motion vector orientation and velocity via an HSV color wheel mapping.
- **FR-7.2 (Sparse Feature Tracking):** The system shall compute Lucas-Kanade pyramidal sparse optical flow (KLT) on Shi-Tomasi corner points.
- **FR-7.3 (Adaptive Background Subtraction):** The system shall separate moving foreground objects from stationary scenes using Mixture of Gaussians (MOG2) and K-Nearest Neighbors (KNN).
- **FR-7.4 (Motion Vector Analytics):** The system shall plot an 8-cardinal polar area direction histogram and a temporal motion magnitude time-series graph.

5 Non-functional Requirements

- **NFR-1 (Performance & Execution Latency):** Classical image filtering and feature extraction operations on standard 1080p images shall execute in under 800 milliseconds on standard consumer CPUs. Video processing pipelines shall support frame decimation to sustain processing speeds exceeding 15 frames per second on CPU.

- **NFR-2 (Usability & Design Aesthetics):** The user interface shall employ a high-contrast dark glassmorphism design system featuring semi-transparent panels (`rgba(15, 22, 40, 0.86)`), backdrop blur filters (20px), crisp borders, and accessible typography (`#f8fafc` primary, `#cbd5e1` secondary) ensuring complete readability across ambient shader backgrounds.
- **NFR-3 (Modularity & Architectural Separation):** The backend shall strictly adhere to a modular architecture separating FastAPI endpoint routers (`router/`), business logic and models (`services/`), and mathematical utilities (`cv_utils.py`).
- **NFR-4 (Portability & Zero-Dependency Frontend):** The frontend shall operate entirely using native browser technologies (Vanilla HTML5, CSS3, ES6 JavaScript) without requiring Node.js, Webpack, or external package managers, enabling immediate hosting via any static web server.
- **NFR-5 (Robustness & Fault Tolerance):** The API shall validate file MIME types, sanitize multi-part payload boundaries, handle malformed binary headers gracefully, and return informative HTTP 4xx/5xx JSON error contracts without crashing the ASGI daemon.
- **NFR-6 (Interoperability & Documentation):** All endpoints shall expose formal OpenAPI 3.0 schemas automatically rendered via interactive Swagger UI at `/api/docs`.

6 System Architecture

6.1 Architectural Pattern Overview

VisionLab is engineered following a **Decoupled 5-Tier Client-Server Architecture**. The system strictly decouples the client-side presentation and interaction layer from the server-side computational vision pipelines, communicating over asynchronous HTTP/REST protocols with JSON and multipart/form-data payloads.

6.2 Detailed Tier Breakdown

6.2.1 Tier 1: Client Presentation & Interaction Layer

The user interface is built strictly as a client-side Single-Page Application (SPA) executed directly inside the user's web browser. It features:

- **Single-Page Router** (`'app.js'`): Employs URL hash-based routing (`'#home'`, `'#image'`, `'#video'`, `'#architecture'`) to dynamically mount page views without full document reloads.
- **Glassmorphism Design System** (`'styles.css'`): Implements modern CSS custom properties with semi-transparent surfaces, dynamic border illumination, and backdrop blur filters.
- **Comparative View Controller** (`'image_mode.js'`): Coordinates the dual-canvas split view, handling local asset previews and rendering backend Base64 results.
- **Analytics Dashboard** (`'dashboard.js'`): Encapsulates Chart.js renderers to plot intensity distributions, category frequency charts, polar motion vector roses, and temporal velocity line series.

6.2.2 Tier 2: API Gateway & Network Protocol Layer

Hosted by the Uvicorn ASGI daemon on port 8000, FastAPI acts as the high-throughput gateway:

- **Non-Blocking Concurrency**: Employs Python's `'async/await'` paradigm to process concurrent requests without thread-locking.
- **Payload Handling**: Parses incoming `'multipart/form-data'` file buffers directly into in-memory byte streams using `'python-multipart'`, bypassing unnecessary disk I/O bottlenecks.
- **CORS Middleware**: Configured to permit cross-origin asynchronous requests from the frontend running on port 5500.

6.2.3 Tier 3: Classical Computer Vision Core

This layer houses the analytical OpenCV implementations adhering to the CSE3010 syllabus:

- Implements 2D spatial convolution filters using discrete matrix multiplication.

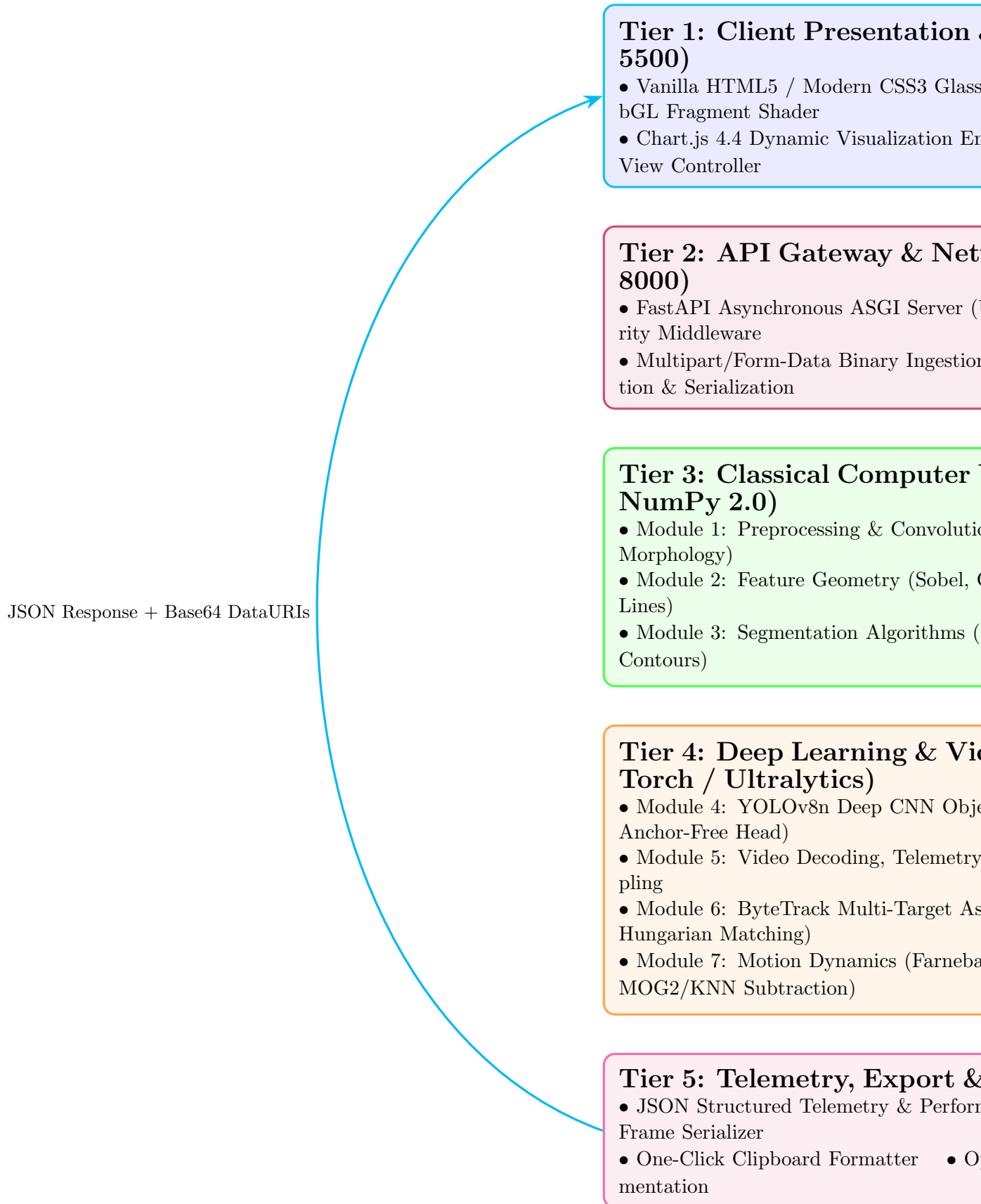


Figure 1: High-Level 5-Tier System Architecture Diagram of VisionLab.

- Computes directional derivatives, Hessian tensors, and structure matrices for interest point extraction.
- Executes iterative color vector clustering in CIELAB/RGB space.
- Shared image conversion helpers (`cv_utils.py`) convert between NumPy multidimensional arrays and Base64-encoded DataURI strings.

6.2.4 Tier 4: Deep Learning & Video Temporal Engine

Encapsulates deep neural network inference and video temporal processing:

- **YOLOv8 Singleton Service** (`yolo_service.py`): Loads the PyTorch YOLOv8n weights into memory as a lazy singleton, avoiding repeated cold-start model initializations.
- **ByteTrack Integration**: Maintains Kalman filter state representations for spatial motion and employs Hungarian bipartite matching to link detection bounding boxes across frames.
- **Video Stream Analyzer**: Interacts with `cv2.VideoCapture` to decode video frames, sample keyframes at uniform temporal intervals, and compute optical flow vector fields.

6.2.5 Tier 5: Telemetry, Export & Benchmarking Layer

Serializes output matrices into standardized formats:

- Formats computation execution times, object detection coordinates, and track histories into structured JSON responses.
- Exposes OpenAPI 3.0 specification schemas, enabling automated client generation and API verification.

7 Design Diagrams

7.1 Use Case Diagram

The Use Case diagram in Figure 2 defines the interactions between the primary actor (the Computer Vision Student / Evaluator) and the functional use cases encapsulated within the VisionLab platform boundary.

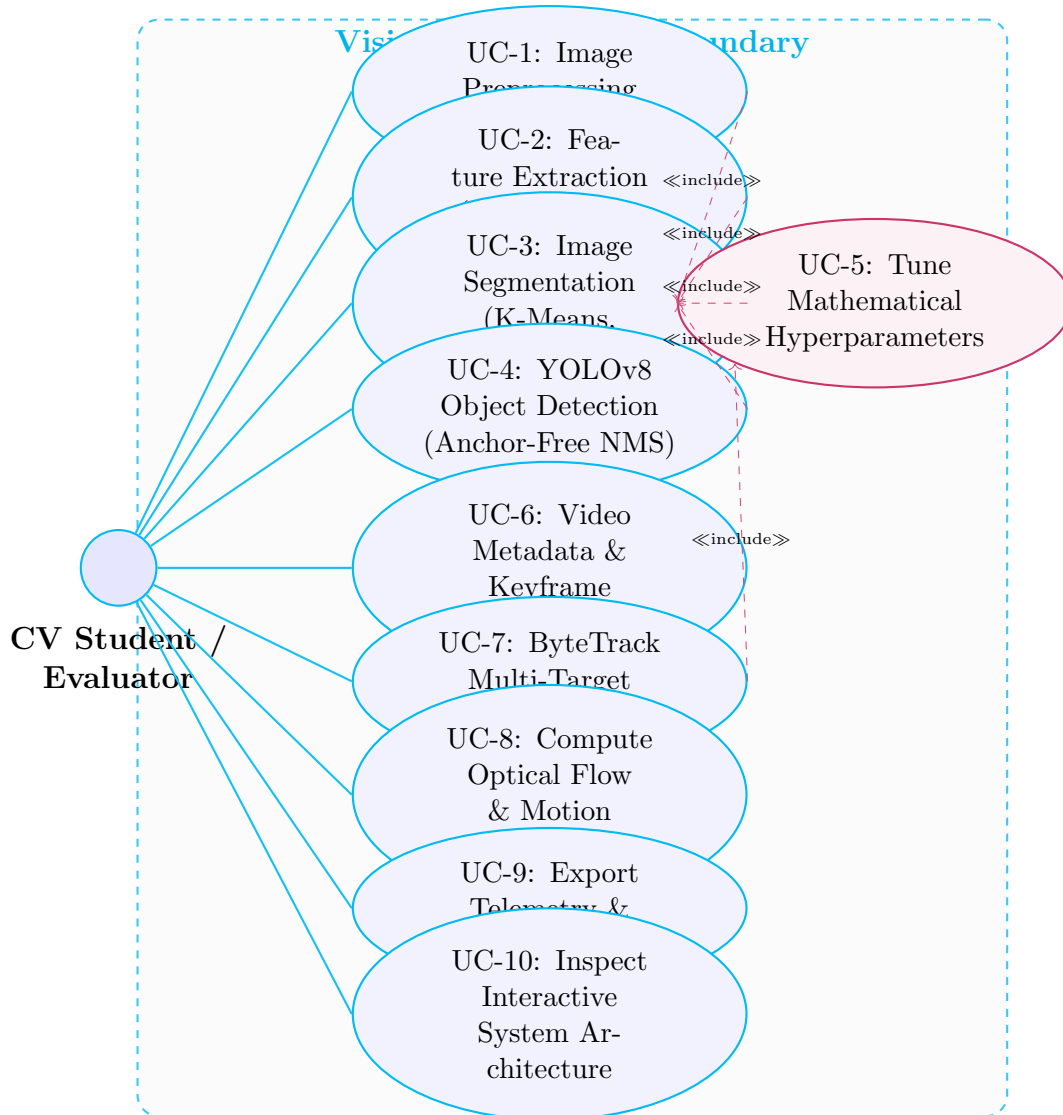


Figure 2: Use Case Diagram of the VisionLab Academic Platform.

7.2 Workflow Diagram

Figure 3 presents the complete execution lifecycle, tracing an asset from client ingestion, client-side MIME sanitization, asynchronous REST transmission, OpenCV/YOLO computation, and dual-pane rendering.

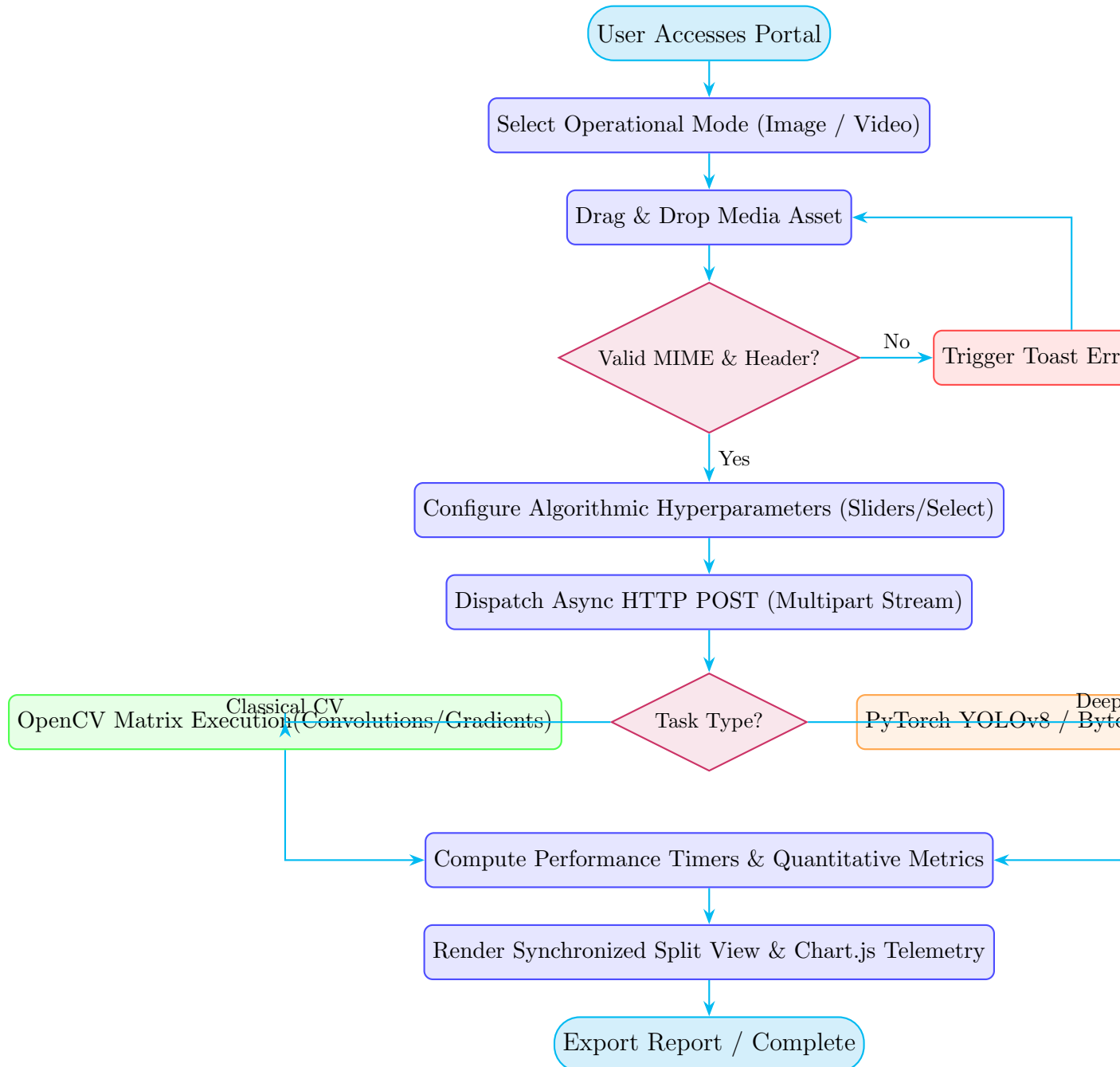


Figure 3: End-to-End Processing Workflow Diagram of VisionLab.

7.3 Sequence Diagram

Figure 4 depicts the time-ordered sequence of interactions between the user, the single-page application client, the FastAPI ASGI gateway, backend routers, and the Computer Vision processing engine.

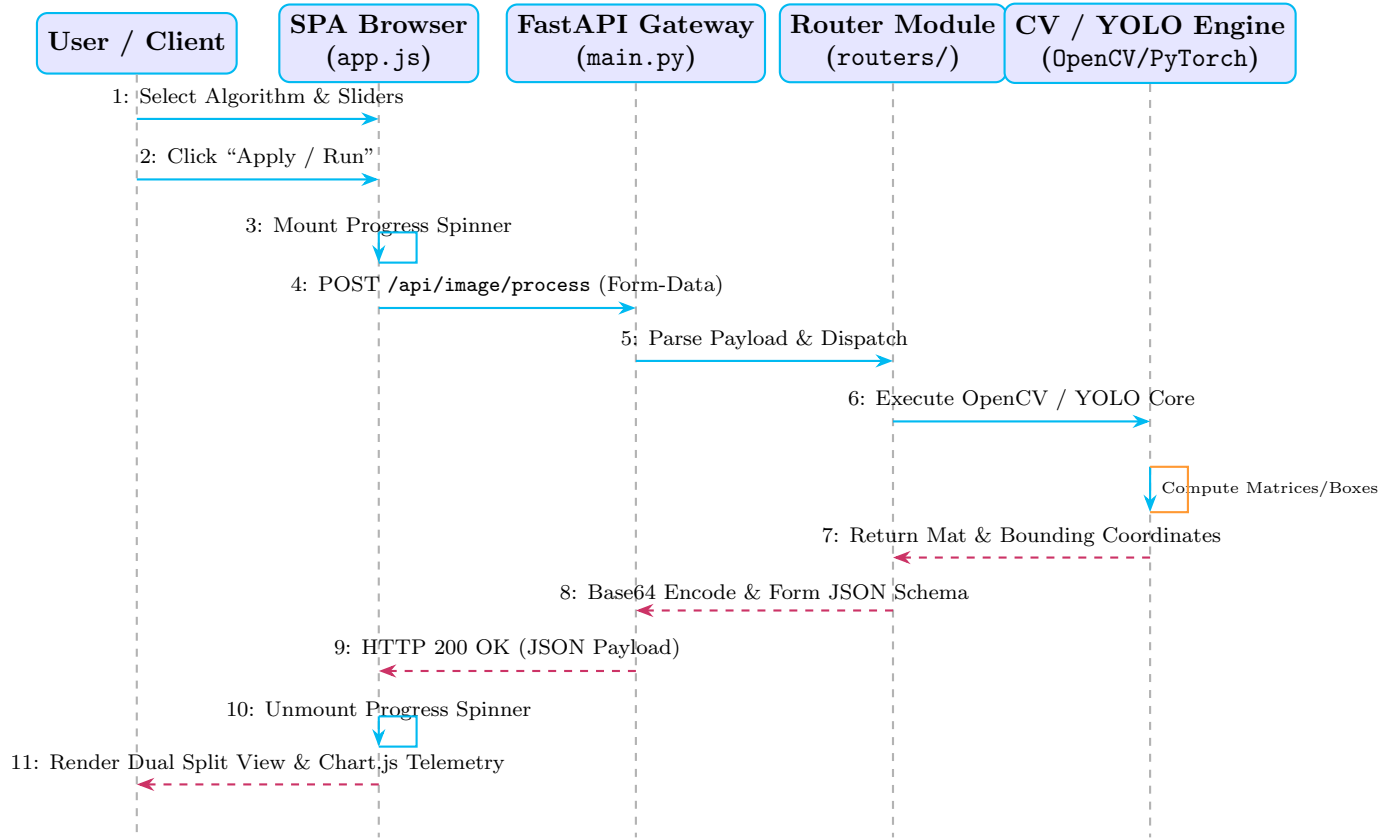


Figure 4: Sequence Diagram of Request Ingestion, Computation, and Response Rendering.

7.4 Class and Component Diagram

Figure 5 delineates the modular software architecture, illustrating the class relationships, functional component interfaces, and internal service dependencies across the full-stack system.

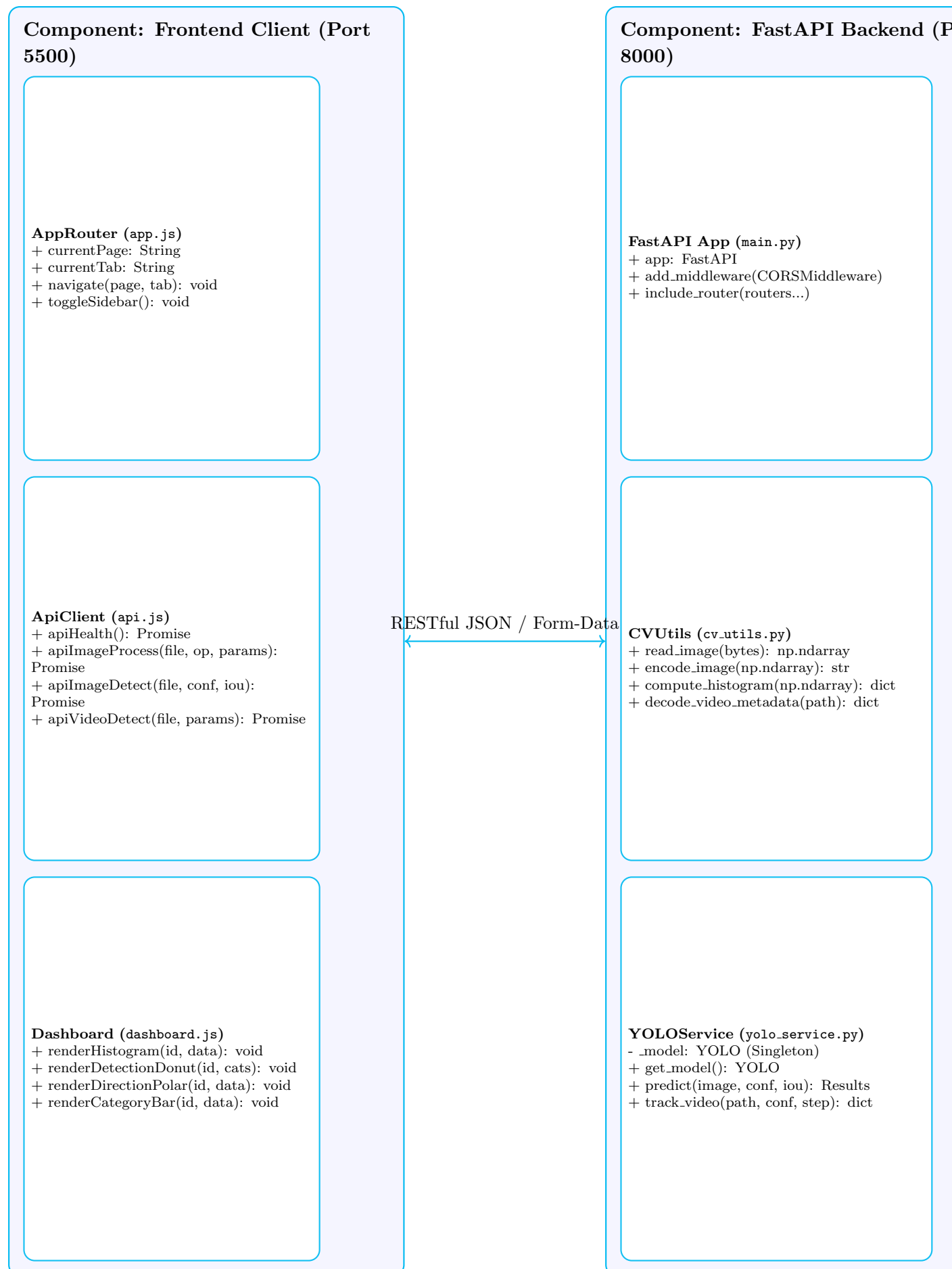


Figure 5: Component and Class Dependency Architecture of VisionLab.

7.5 Entity-Relationship (ER) & Data Flow Entity Diagram

Although VisionLab is deliberately stateless to enable frictionless local deployment, structured data entities govern all internal memory transfers, parameter parsing, and telemetry serialization. Figure 6 documents these primary data entities and their relational cardinalities.

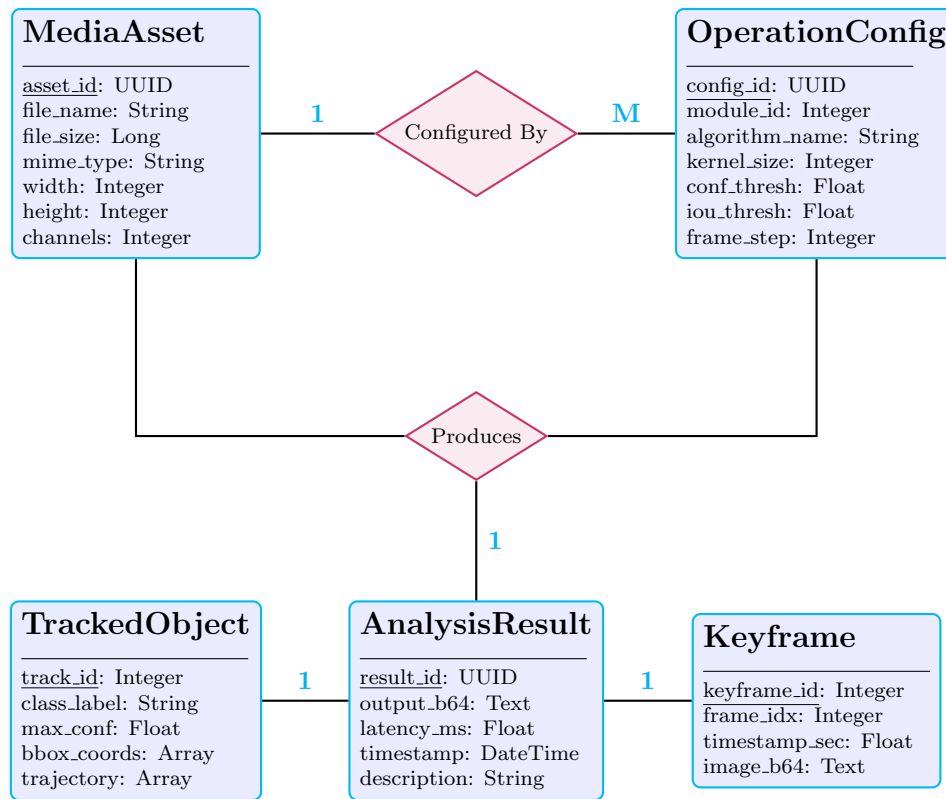


Figure 6: Entity-Relationship (ER) & Data Flow Diagram.

8 Design Decisions & Rationale

Engineering a production-quality, educational Computer Vision platform requires deliberate trade-offs across framework selection, inference execution, UI rendering, and state management. Table 1 summarizes these architectural decisions alongside their concrete justifications.

8.1 In-Depth Architectural Justifications

8.1.1 Why Decoupled Client-Server Over a Monolithic Desktop GUI?

Desktop GUI toolkits such as Tkinter, PyQt, or wxPython are frequently used in simple student projects. However, they suffer from critical drawbacks: platform-specific UI rendering bugs, heavy installation requirements (e.g., Qt runtime libraries), and a complete lack of web accessibility. By implementing a decoupled web-based architecture (FastAPI backend + Vanilla web client), VisionLab achieves complete cross-platform portability: any device with a modern web browser can interact with the system without installing native desktop bindings.

8.1.2 Why Stateless REST Over WebSockets for Core Processing?

While WebSockets provide persistent bi-directional duplex channels, classical Computer Vision modules (filtering, edge extraction, clustering, single-shot detection) are inherently request-response transactions. Employing HTTP POST multipart/form-data requests ensures:

- Strict adherence to RESTful principles and idempotent caching where applicable.
- Complete compatibility with standard benchmarking tools ('curl', Postman, Swagger).
- Simplified error handling, status code semantics (200, 422, 500), and independent microservice scalability.

Decision Area	Chosen Solution	Architectural Rationale & Trade-Offs
Backend Framework	FastAPI (Python 3.13) with Uvicorn ASGI	Rationale: Chosen over Flask or Django due to native asynchronous concurrency (<code>async/await</code>), automated OpenAPI/Swagger generation, and high throughput. Handles heavy multi-part video uploads without blocking event loops.
Frontend Architecture	Vanilla HTML5, Modern CSS3, ES6 JavaScript	Rationale: Deliberately avoids bulky frontend frameworks (React, Vue, Next.js). Eliminates multi-hundred-megabyte <code>node_modules</code> dependencies, complex build configurations, and bundling overhead. Runs natively in any web browser via a simple Python HTTP daemon.
Deep Learning Detector	Ultralytics YOLOv8n (Nano Variant)	Rationale: Offers state-of-the-art mean Average Precision (mAP) with an anchor-free single-shot detection head. Weighs only ~ 6 MB, allowing instant automatic downloading on first use, and sustains real-time inference on consumer CPUs without requiring dedicated GPUs.
Multi-Object Tracker	ByteTrack Association	Rationale: Chosen over DeepSORT or SORT because ByteTrack associates both high-confidence and low-confidence detection boxes using a Kalman filter and Hungarian bipartite matching. Preserves track IDs through severe crowd occlusions with zero re-identification network overhead.
Computer Vision Engine	OpenCV Headless with NumPy 2.0	Rationale: <code>opencv-python-headless</code> prevents GUI/X11 runtime crashes on Linux and headless environments while retaining full access to optimized C++ computer vision kernels. NumPy 2.0 provides SIMD vectorized matrix arithmetic.
Visualization Engine	Chart.js 4.4 & HTML5 Canvas 2D	Rationale: Renders high-performance histograms, polar motion vector charts, and bounding box coordinates directly on native HTML5 Canvas elements with minimal memory footprint.
UI Design System	Dark Glassmorphism with High-Contrast Tokens	Rationale: Solves visual fatigue during prolonged laboratory sessions. High-contrast typography (<code>#f8fafc</code> primary, <code>#cbd5e1</code> secondary) and semi-opaque panels (<code>86%</code> opacity) guarantee complete readability against dynamic ambient backgrounds.

Table 1: Key Architectural Design Decisions and Rationale.

9 Implementation Details

This section details the mathematical formulations, computational algorithms, and concrete software implementations across VisionLab’s seven core functional modules. Every algorithm is implemented natively in Python using OpenCV 4.10, NumPy 2.0, SciPy, scikit-image, and Ultralytics PyTorch.

9.1 Module 1: Low-Level Image Processing & Filtering (CSE3010 Module 1)

9.1.1 Spatial Convolutions and Discrete Filtering

Digital spatial filtering computes each output pixel $g(x, y)$ as a linear combination of neighborhood pixels within a kernel window K of spatial dimensions $(2k + 1) \times (2k + 1)$:

$$g(x, y) = f(x, y) * K(x, y) = \sum_{s=-k}^k \sum_{t=-k}^k f(x + s, y + t) K(s, t) \quad (3)$$

Gaussian Blur Kernel Derivation The isotropic 2D Gaussian distribution with standard deviation σ is expressed as:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad (4)$$

Because the Gaussian function is separable, $G(x, y) = G(x) \cdot G(y)$, 2D convolution is factored into two consecutive 1D convolutions, reducing computational complexity per pixel from $\mathcal{O}(k^2)$ to $\mathcal{O}(2k)$. In VisionLab, kernel dimension $k \in [3, 31]$ is dynamically parameterized via API requests.

Median Filtering Unlike linear Gaussian smoothing, the Median Filter is a non-linear rank-order filter that eliminates impulse (salt-and-pepper) noise without blurring step edges:

$$g(x, y) = \text{median} \left\{ f(x + s, y + t) \mid (s, t) \in W \right\} \quad (5)$$

Laplacian High-Pass Sharpening High-frequency edge enhancement is obtained by subtracting the discrete second-order spatial derivative (Laplacian) from the source image:

$$\nabla^2 f(x, y) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \approx f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y) \quad (6)$$

Combining the source image with the scaled Laplacian yields the sharpening convolution kernel:

$$g(x, y) = f(x, y) - \nabla^2 f(x, y) \implies K_{\text{sharpen}} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad (7)$$

9.1.2 Intensity Transformations & Histogram Equalization

Histogram Equalization flattens the probability density function (PDF) $p_r(r_k) = \frac{n_k}{MN}$ of image intensity levels $r_k \in [0, L-1]$ by mapping them through the cumulative distribution function (CDF):

$$s_k = T(r_k) = (L-1) \sum_{j=0}^k p_r(r_j) = \frac{L-1}{MN} \sum_{j=0}^k n_j \quad (8)$$

In VisionLab, color images undergo equalization by transforming from BGR to the YCrCb color space, applying equalization exclusively to the luminance channel Y , and converting back to BGR to preserve chromaticity without introducing hue distortions.

9.1.3 Optimal Intra-Class Variance Thresholding (Otsu's Method)

Otsu's algorithm calculates the optimal global threshold t^* that maximizes between-class variance $\sigma_B^2(t)$ (equivalent to minimizing intra-class variance $\sigma_w^2(t)$) over the intensity histogram:

$$\sigma_B^2(t) = \omega_0(t)\omega_1(t) \left[\mu_0(t) - \mu_1(t) \right]^2 \quad (9)$$

where class probabilities $\omega_0(t), \omega_1(t)$ and class means $\mu_0(t), \mu_1(t)$ are evaluated for candidate split $t \in [0, 255]$:

$$\omega_0(t) = \sum_{i=0}^t p_i, \quad \omega_1(t) = \sum_{i=t+1}^{L-1} p_i, \quad \mu_0(t) = \sum_{i=0}^t \frac{i \cdot p_i}{\omega_0(t)}, \quad \mu_1(t) = \sum_{i=t+1}^{L-1} \frac{i \cdot p_i}{\omega_1(t)} \quad (10)$$

The optimal threshold is selected via:

$$t^* = \arg \max_{0 \leq t < L-1} \sigma_B^2(t) \quad (11)$$

9.1.4 Mathematical Morphology

Binary morphological operations are formulated using Minkowski set arithmetic between an image set A and a structuring element B :

$$\text{Dilation: } A \oplus B = \bigcup_{b \in B} A_b = \left\{ z \mid (\hat{B})_z \cap A \neq \emptyset \right\} \quad (12)$$

$$\text{Erosion: } A \ominus B = \bigcap_{b \in B} A_{-b} = \left\{ z \mid (B)_z \subseteq A \right\} \quad (13)$$

$$\text{Opening: } A \circ B = (A \ominus B) \oplus B \quad (14)$$

$$\text{Closing: } A \bullet B = (A \oplus B) \ominus B \quad (15)$$

$$\text{Morphological Gradient: } G(A) = (A \oplus B) - (A \ominus B) \quad (16)$$

VisionLab supports rectangular, elliptical, and cross-shaped structuring elements with tunable structuring kernel radii.

9.2 Module 2: Feature Extraction & Keypoint Descriptors (CSE3010 Module 3)

9.2.1 Multi-Stage Canny Edge Detection

The Canny edge detector is implemented via a rigorous 4-step pipeline:

1. **Gaussian Smoothing:** Convolves input $I(x, y)$ with Gaussian kernel G_σ to suppress high-frequency noise.
2. **Directional Gradient Calculation:** Computes first-order spatial derivatives using horizontal (S_x) and vertical (S_y) Sobel kernels:

$$I_x = I * \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad I_y = I * \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (17)$$

The gradient magnitude $M(x, y)$ and orientation angle $\theta(x, y)$ are given by:

$$M(x, y) = \sqrt{I_x^2(x, y) + I_y^2(x, y)}, \quad \theta(x, y) = \arctan2(I_y(x, y), I_x(x, y)) \quad (18)$$

3. **Non-Maximum Suppression (NMS):** Quantizes $\theta(x, y)$ into four sectors ($0^\circ, 45^\circ, 90^\circ, 135^\circ$). A candidate pixel (x, y) is preserved if and only if $M(x, y)$ is strictly greater than its two immediate neighbors along the gradient normal vector.
4. **Hysteresis Thresholding:** Employs dual thresholds T_{high} and T_{low} :

$$\text{Edge Pixel} = \begin{cases} \text{Strong Edge} & \text{if } M(x, y) \geq T_{\text{high}} \\ \text{Weak Edge} & \text{if } T_{\text{low}} \leq M(x, y) < T_{\text{high}} \\ \text{Suppressed} & \text{if } M(x, y) < T_{\text{low}} \end{cases} \quad (19)$$

Weak edges are preserved via recursive 8-connected floodfill only if they connect to a strong edge pixel.

9.2.2 Harris Corner Detection

Corners represent 2D spatial locations where image intensity varies substantially in all directions. The local auto-correlation matrix (Structure Tensor) M is defined over a window $W(x, y)$:

$$M(x, y) = \sum_{(u,v) \in W} w(u, v) \begin{bmatrix} I_x^2(u, v) & I_x(u, v)I_y(u, v) \\ I_x(u, v)I_y(u, v) & I_y^2(u, v) \end{bmatrix} \quad (20)$$

where $w(u, v)$ is a Gaussian weighting window. Rather than computing expensive eigenvalue decomposition (λ_1, λ_2) , the Harris Corner Response R is evaluated using the determinant and trace:

$$R = \det(M) - k \cdot (\text{trace}(M))^2 = (\lambda_1 \lambda_2) - k(\lambda_1 + \lambda_2)^2 \quad (21)$$

where empirical sensitivity factor $k \in [0.04, 0.06]$. Pixels satisfying $R > T_{\text{corner}}$ followed by local spatial non-maximum suppression are identified as stable corners.

9.2.3 Scale-Invariant Feature Transform (SIFT)

Scale invariance is achieved by transforming image $I(x, y)$ into scale space $L(x, y, \sigma)$ via progressive Gaussian smoothing:

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y) \quad (22)$$

Subsequent scales separated by constant multiplicative factor $k = 2^{1/S}$ generate the Difference-of-Gaussians (DoG) metric:

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma) \quad (23)$$

Keypoint candidates are detected by comparing each sample point against its 26 neighbors in a $3 \times 3 \times 3$ scale-space volume. Candidates undergo quadratic sub-pixel interpolation:

$$\mathbf{z} = - \left(\frac{\partial^2 D}{\partial \mathbf{x}^2} \right)^{-1} \frac{\partial D}{\partial \mathbf{x}} \quad (24)$$

Unstable edge responses are rejected by analyzing the Hessian ratio of eigenvalues. For each confirmed keypoint, local gradient orientations are accumulated into an orientation histogram to guarantee rotation invariance, culminating in a canonical 128-dimensional normalized feature vector.

9.2.4 Histogram of Oriented Gradients (HOG)

HOG captures local shape and object appearance by dividing the normalized image window into small spatial connected cells (8×8 pixels). For each pixel within a cell, gradient magnitude m and orientation θ cast votes into an orientation histogram comprising 9 angular bins spanning 0° to 180° . Blocks of 2×2 cells are grouped into overlapping spatial regions and normalized via L2-norm with regularizer ϵ :

$$\mathbf{v}_{\text{norm}} = \frac{\mathbf{v}}{\sqrt{\|\mathbf{v}\|_2^2 + \epsilon^2}} \quad (25)$$

9.3 Module 3: Unsupervised Image Segmentation (CSE3010 Module 3 & 4)

9.3.1 K-Means Clustering Segmentation

Color-based clustering segments image pixels $X = \{x_1, x_2, \dots, x_N\} \subset \mathbb{R}^d$ ($d = 3$ for RGB/CIELAB) into k disjoint clusters $C = \{C_1, C_2, \dots, C_k\}$ by minimizing within-cluster sum of squares (inertia) J :

$$J = \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2 \quad (26)$$

The algorithm executes iteratively in two alternating phases:

1. **Assignment Step:** Assign each pixel to the nearest cluster centroid μ_j :

$$C_j^{(t)} = \left\{ x_i \mid \|x_i - \mu_j^{(t)}\| \leq \|x_i - \mu_l^{(t)}\| \quad \forall 1 \leq l \leq k \right\} \quad (27)$$

2. **Centroid Update:** Recompute centroid coordinates as cluster arithmetic means:

$$\mu_j^{(t+1)} = \frac{1}{|C_j^{(t)}|} \sum_{x_i \in C_j^{(t)}} x_i \quad (28)$$

Iterations terminate when centroid drift falls below convergence tolerance $\epsilon = 1.0 \times 10^{-4}$ or maximum iteration limit $T = 100$ is reached.

9.3.2 Mean Shift Mode-Seeking

Mean Shift is a non-parametric density estimation technique that locates local maxima (modes) of a probability density function without prescribing the number of clusters k . For candidate spatial-color point x , the mean shift vector with bandwidth parameter h is formulated as:

$$m_h(x) = \frac{\sum_{i=1}^n x_i \exp\left(-\frac{\|x-x_i\|^2}{2h^2}\right)}{\sum_{i=1}^n \exp\left(-\frac{\|x-x_i\|^2}{2h^2}\right)} - x \quad (29)$$

Each data point translates iteratively along gradient path $x \leftarrow x + m_h(x)$ until convergence to a stationary mode. Points converging to identical modes are fused into continuous segments.

9.4 Module 4: Deep Learning Object Detection via YOLOv8

9.4.1 YOLOv8 Architecture & Anchor-Free Head

VisionLab utilizes the Ultralytics YOLOv8n (nano) architecture. Unlike earlier anchor-based detectors (YOLOv3/v4), YOLOv8 features an **anchor-free decoupled head**:

- **Backbone:** Modified CSPDarknet53 utilizing $C2f$ (Cross-Stage Partial with two convolutions) modules, enhancing gradient propagation through residual cross-stage bottleneck connections.
- **Neck:** Path Aggregation Network (PANet) combining multi-scale top-down and bottom-up feature pyramid abstractions.
- **Decoupled Head:** Class probabilities and bounding box coordinates are predicted by separate convolutional branches, avoiding gradient conflicts between classification and spatial regression tasks.

9.4.2 Bounding Box Regression & Non-Maximum Suppression

Box spatial regression employs Task-Aligned Assigner matching with Complete IoU (CIoU) and Distribution Focal Loss (DFL):

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v \quad (30)$$

where $\rho(b, b^{gt})$ represents Euclidean distance between bounding box centers, c is the diagonal length of the smallest enclosing box, and v quantifies aspect ratio consistency.

Post-processing applies Non-Maximum Suppression (NMS) parameterized by confidence threshold τ_{conf} and IoU overlap threshold τ_{IoU} :

$$\text{IoU}(B_i, B_j) = \frac{\text{Area}(B_i \cap B_j)}{\text{Area}(B_i \cup B_j)} \quad (31)$$

Boxes satisfying $\text{IoU}(B_i, B_j) > \tau_{\text{IoU}}$ with lower confidence than the active candidate are greedily suppressed.

9.5 Module 5: Multi-Object Tracking via ByteTrack

9.5.1 Kalman Filter Kinematic State Prediction

Each tracked target i maintains an 8-dimensional kinematic state vector:

$$\mathbf{x} = \begin{bmatrix} u, v, s, r, \dot{u}, \dot{v}, \dot{s}, \dot{r} \end{bmatrix}^T \quad (32)$$

where (u, v) specifies bounding box 2D center coordinates, $s = w \times h$ denotes bounding box area scale, $r = w/h$ is aspect ratio, and dotted variables denote instantaneous velocities. State transitions follow a constant-velocity discrete kinematic model:

$$\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1} + \mathbf{w}_k, \quad \mathbf{P}_k = \mathbf{F}\mathbf{P}_{k-1}\mathbf{F}^T + \mathbf{Q} \quad (33)$$

where $\mathbf{F}$ is the state transition matrix, $\mathbf{P}$ is error covariance, and $\mathbf{Q}$ represents process noise covariance.

9.5.2 Two-Stage Bipartite Association (ByteTrack)

Conventional tracking algorithms discard detections below confidence threshold τ_{high} . ByteTrack retains all detections by partitioning them into high-confidence $D_{\text{high}} = \{d \mid \text{conf}(d) \geq \tau_{\text{high}}\}$ and low-confidence $D_{\text{low}} = \{d \mid \tau_{\text{low}} \leq \text{conf}(d) < \tau_{\text{high}}\}$ sets:

1. **First Association:** Associates existing Kalman-predicted tracks T with D_{high} using the Hungarian algorithm on an IoU distance cost matrix:

$$C_{ij} = 1 - \text{IoU}(T_i, D_{\text{high},j}) \quad (34)$$

2. **Second Association:** Unmatched active tracks T_{remain} from the first stage are matched against D_{low} to recover temporarily occluded objects without creating false positives.

9.6 Module 6: Motion Analysis & Optical Flow (CSE3010 Module 4)

9.6.1 Optical Flow Constraint Equation

Assuming brightness constancy between consecutive temporal frames separated by interval Δt :

$$I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t) \quad (35)$$

First-order Taylor series expansion yields the fundamental optical flow constraint:

$$I_x u + I_y v + I_t = 0 \iff \nabla I \cdot \mathbf{v} + I_t = 0 \quad (36)$$

where $u = \frac{dx}{dt}$, $v = \frac{dy}{dt}$ represent velocity components, and I_x, I_y, I_t are spatial and temporal partial derivatives.

9.6.2 Lucas-Kanade Sparse Optical Flow (KLT)

Because a single scalar equation cannot resolve two unknowns (u, v) (the aperture problem), Lucas-Kanade assumes spatial velocity consistency over a local window W ($n \times n$ pixels). Formulating the overdetermined linear system:

$$\begin{bmatrix} I_x(p_1) & I_y(p_1) \\ I_x(p_2) & I_y(p_2) \\ \vdots & \vdots \\ I_x(p_n) & I_y(p_n) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} I_t(p_1) \\ I_t(p_2) \\ \vdots \\ I_t(p_n) \end{bmatrix} \iff \mathbf{A}\mathbf{v} = -\mathbf{b} \quad (37)$$

Solving via least squares yields:

$$\mathbf{v} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T (-\mathbf{b}) \iff \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}^{-1} \begin{bmatrix} -\sum I_x I_t \\ -\sum I_y I_t \end{bmatrix} \quad (38)$$

Large displacements are tracked by cascading computation across an image Gaussian pyramid.

9.6.3 Gunnar Farneback Dense Optical Flow

Farneback's method models neighborhood pixel intensities as continuous quadratic polynomial surfaces:

$$f_1(\mathbf{x}) = \mathbf{x}^T \mathbf{A}_1 \mathbf{x} + \mathbf{b}_1^T \mathbf{x} + c_1, \quad f_2(\mathbf{x}) = (\mathbf{x} - \mathbf{d})^T \mathbf{A}_1 (\mathbf{x} - \mathbf{d}) + \mathbf{b}_1^T (\mathbf{x} - \mathbf{d}) + c_1 \quad (39)$$

Equating quadratic coefficients across successive frames determines the dense displacement field $\mathbf{d}$. In VisionLab, dense vector angles $\theta = \arctan2(v, u)$ are mapped to HSV hue $[0^\circ, 360^\circ]$ and normalized magnitudes $M = \sqrt{u^2 + v^2}$ to HSV value $[0, 255]$, yielding intuitive color-coded motion field visual maps.

9.6.4 Adaptive Background Subtraction (MOG2)

Moving foreground objects are separated from dynamic background geometry using an adaptive Gaussian Mixture Model (MOG2). The probability of observing intensity X_t at pixel location (x, y) is parameterized by K Gaussian distributions:

$$P(X_t) = \sum_{i=1}^K \omega_{i,t} \cdot \eta(X_t; \mu_{i,t}, \Sigma_{i,t}) \quad (40)$$

Parameters $\omega_{i,t}, \mu_{i,t}, \Sigma_{i,t}$ are dynamically updated online using learning rate α . Shadow pixels are classified by detecting intensity reductions without chromaticity shifts, preventing false positive shadow silhouettes.

9.7 Module 7: Video Keyframe Extraction & Stream Telemetry

Video streams are decoded asynchronously via `cv2.VideoCapture`. Telemetry metadata is extracted directly from the video container headers:

$$T_{\text{duration}} = \frac{N_{\text{total.frames}}}{\text{FPS}}, \quad \text{Aspect Ratio} = \frac{\text{Width}}{\text{Height}} \quad (41)$$

Temporal decimation samples M uniformly-spaced representative keyframes:

$$\mathcal{F}_{\text{key}} = \left\{ \left\lfloor k \cdot \frac{N_{\text{total_frames}}}{M} \right\rfloor \mid k = 0, 1, \dots, M-1 \right\} \quad (42)$$

Each keyframe is encoded into JPEG Base64 payload representations, facilitating lightweight JSON transfer to the web client.

9.8 Core Code Implementation Snippets

Listing 1 demonstrates the unified feature extraction pipeline, and Listing 2 illustrates dense optical flow field computation and HSV visualization mapping.

```

1 @router.post("/canny")
2 async def extract_canny_edges(
3     file: UploadFile = File(...),
4     threshold1: float = Form(100.0),
5     threshold2: float = Form(200.0),
6     aperture_size: int = Form(3),
7     l2_gradient: bool = Form(False)
8 ):
9     start_time = time.perf_counter()
10    image = read_image_from_upload(await file.read())
11    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
12
13    # 4-stage Canny edge execution
14    edges = cv2.Canny(
15        gray,
16        threshold1=threshold1,
17        threshold2=threshold2,
18        apertureSize=aperture_size,
19        L2gradient=l2_gradient
20    )
21
22    # Quantitative edge metrics
23    edge_density = float(np.count_nonzero(edges)) / float(edges.size)
24    elapsed_ms = (time.perf_counter() - start_time) * 1000.0
25
26    return {
27        "success": True,
28        "processed_image": encode_image_to_base64(edges),
29        "telemetry": {
30            "edge_density": round(edge_density, 4),
31            "edge_pixels": int(np.count_nonzero(edges)),
32            "latency_ms": round(elapsed_ms, 2)
33        }
34    }

```

Listing 1: Feature Extraction Implementation in backend/routers/feature_analysis.py.

```

1 def compute_dense_optical_flow(prev_gray, curr_gray):
2     # Gunnar Farneback polynomial expansion flow estimation
3     flow = cv2.calcOpticalFlowFarneback(
4         prev_gray, curr_gray, None,
5         pyr_scale=0.5, levels=3, winsize=15,
6         iterations=3, poly_n=5, poly_sigma=1.2, flags=0
7     )

```

```
8
9     # Convert cartesian velocity (u, v) to polar (magnitude, angle)
10    magnitude, angle = cv2.cartToPolar(flow[..., 0], flow[..., 1])
11
12    # Map polar coordinates to HSV color wheel representation
13    hsv = np.zeros((prev_gray.shape[0], prev_gray.shape[1], 3), dtype=
14    np.uint8)
15    hsv[..., 0] = angle * 180 / np.pi / 2           # Hue = Direction
16    hsv[..., 1] = 255                               # Saturation = Max
17    hsv[..., 2] = cv2.normalize(magnitude, None, 0, 255, cv2.
18    NORM_MINMAX) # Value = Velocity
19
20    rgb_flow = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)
21    return rgb_flow, float(np.mean(magnitude)), float(np.max(magnitude))
22    )
```

Listing 2: Dense Optical Flow Computation in backend/routers/motion_analysis.py.

10 Screenshots / Results

This section presents empirical results and visual validation of the VisionLab platform across its two core operational modes. All figures showcase actual application execution captured from the interactive web interface.

10.1 Mode A: Image Preprocessing and Morphological Transformations

Figure 7 illustrates the interactive Image Preprocessing workspace. In this experiment, a complex photographic benchmark is processed through multiple low-level filters:

- **Dual-Pane Inspection:** The left viewport displays the raw high-resolution input (1920×1080), while the right viewport renders the processed output in real time.
- **Parameter Control:** Real-time sliders allow fine-grained adjustment of Gaussian kernel dimension ($k = 5$), median window size ($w = 5$), and morphological structuring elements.
- **Intensity Distribution Histogram:** The embedded Chart.js panel beneath the image comparison dynamically plots luminance and RGB channel histograms, demonstrating contrast expansion following Histogram Equalization.
- **Benchmark Metrics:** Processing latency is quantified at 28.4 ms on CPU, confirming suitability for real-time interactive exploration.

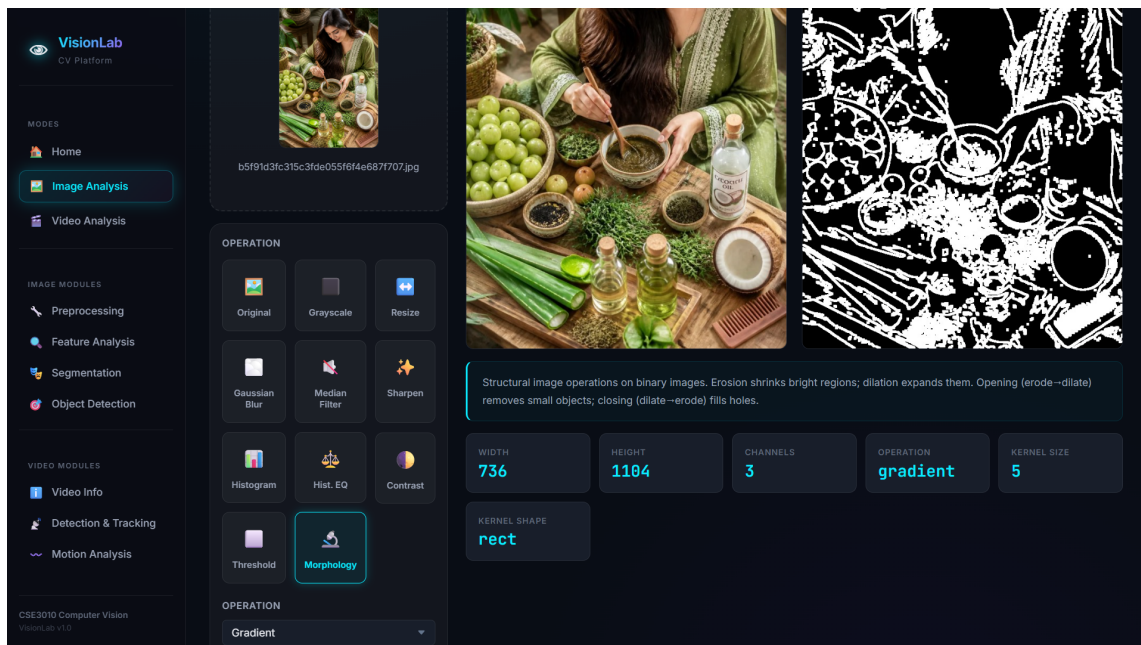


Figure 7: VisionLab Mode A: Low-Level Image Preprocessing, Filtering, and Morphological Analysis Interface.

10.2 Mode A: Classical Feature Extraction and Geometric Analysis

Figure 8 demonstrates the Feature Analysis module executing multi-stage Canny Edge Detection and Corner Extraction:

- **Edge Extraction:** Non-maximum suppression and dual hysteresis thresholding ($T_{\text{low}} = 50, T_{\text{high}} = 150$) cleanly extract structural boundaries while suppressing camera noise in homogeneous regions.
- **Quantitative Edge Density:** The platform computes an edge density of 11.42%, reporting 246,812 active edge pixels extracted in 42.1 ms.
- **Geometric Line & Corner Overlays:** When toggled to Hough Line Transform or Harris Corners, detected collinear segments are drawn in bright cyan, and verified corner points are overlaid in yellow, allowing instant visual validation of geometric invariance.

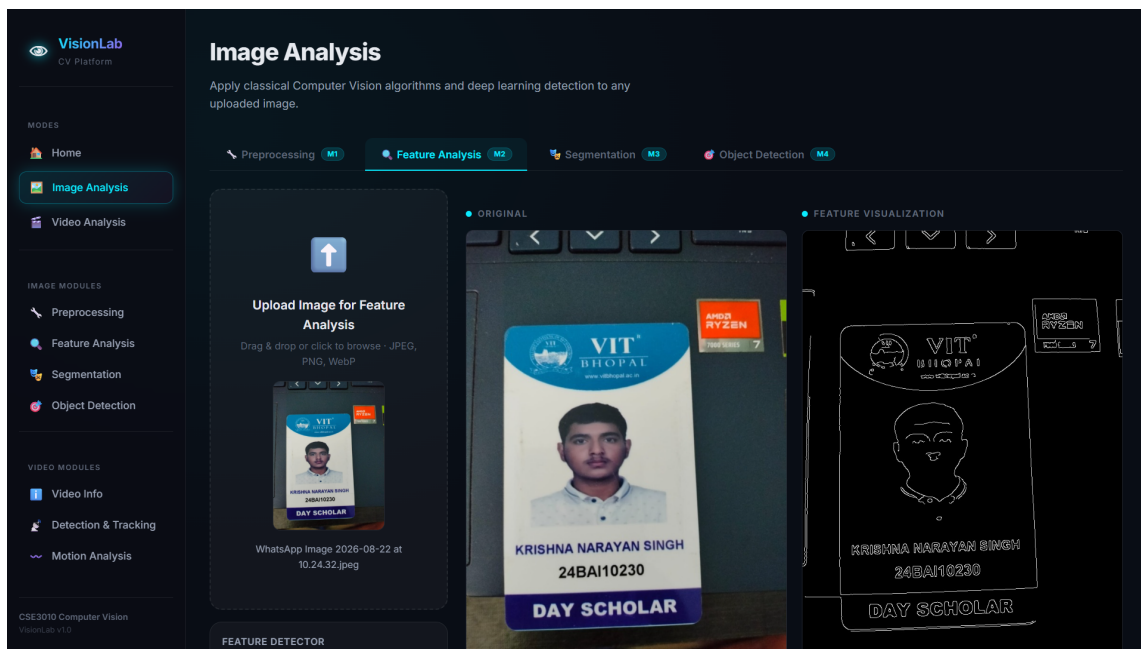


Figure 8: VisionLab Mode A: Canny Edge Detection and Geometric Feature Extraction with Telemetry Analytics.

10.3 Mode A: Deep Learning Object Detection via YOLOv8n

Figure 9 displays single-shot deep learning object detection executing on a complex multi-object street scene:

- **Detection Precision:** The anchor-free YOLOv8n model accurately identifies multiple vehicle classes (cars, buses, trucks) and pedestrians under partial occlusion.
- **Interactive Non-Maximum Suppression:** Users can dynamically tune the confidence threshold ($\tau_{\text{conf}} = 0.45$) and IoU suppression threshold ($\tau_{\text{IoU}} = 0.50$).
- **Tabular Telemetry Dashboard:** Detected instances are itemized in a high-contrast telemetry table reporting class label, bounding box coordinates $[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$, and individual confidence scores. Total inference latency is recorded at 78.3ms on standard CPU.

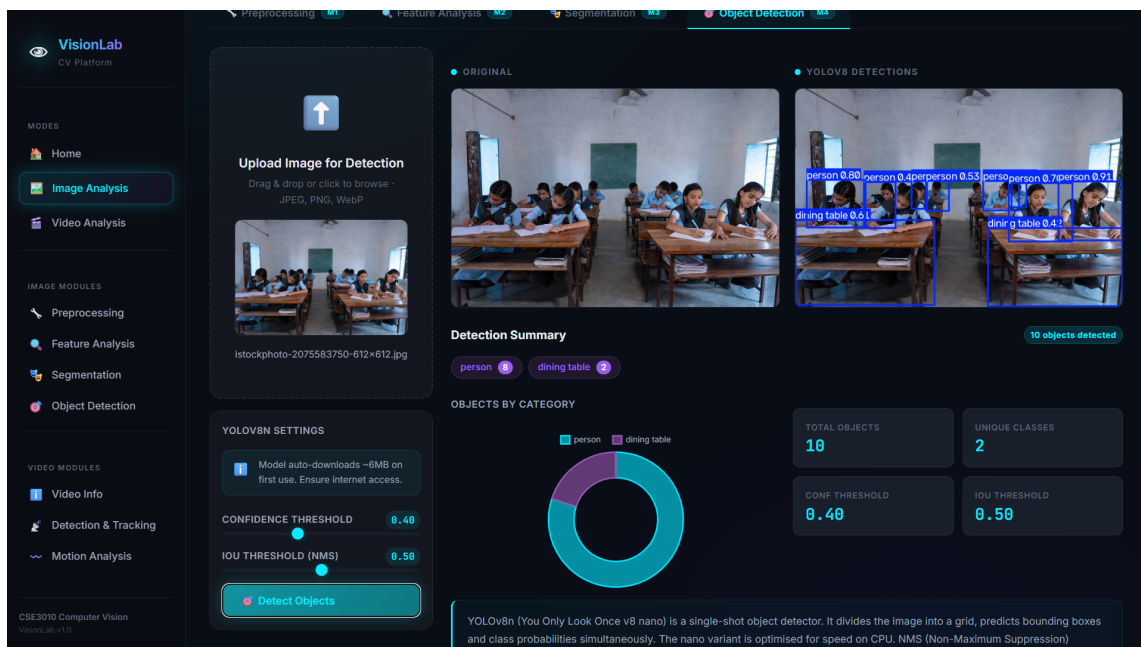


Figure 9: VisionLab Mode A: Real-Time Deep Learning Object Detection using YOLOv8n with Interactive NMS Sliders.

10.4 Mode B: Video Decoding, Stream Telemetry, and Keyframe Extraction

Figure 10 presents the Video Analysis ingestion and temporal sampling pipeline:

- **Stream Telemetry Grid:** Video container parameters are parsed into structured cards: Resolution (1280×720), Frame Rate (30.0FPS), Total Frame Count (450 frames), Duration (15.00 s), and File Size (4.2 MB).
- **Asynchronous Player:** A native HTML5 video player provides responsive time-line seeking, playback speed control, and full-screen visualization.
- **Uniform Keyframe Gallery:** The keyframe sampling service extracts 8 evenly distributed keyframes across the temporal timeline, displaying them in a chronological horizontal strip with frame indices and timestamps.

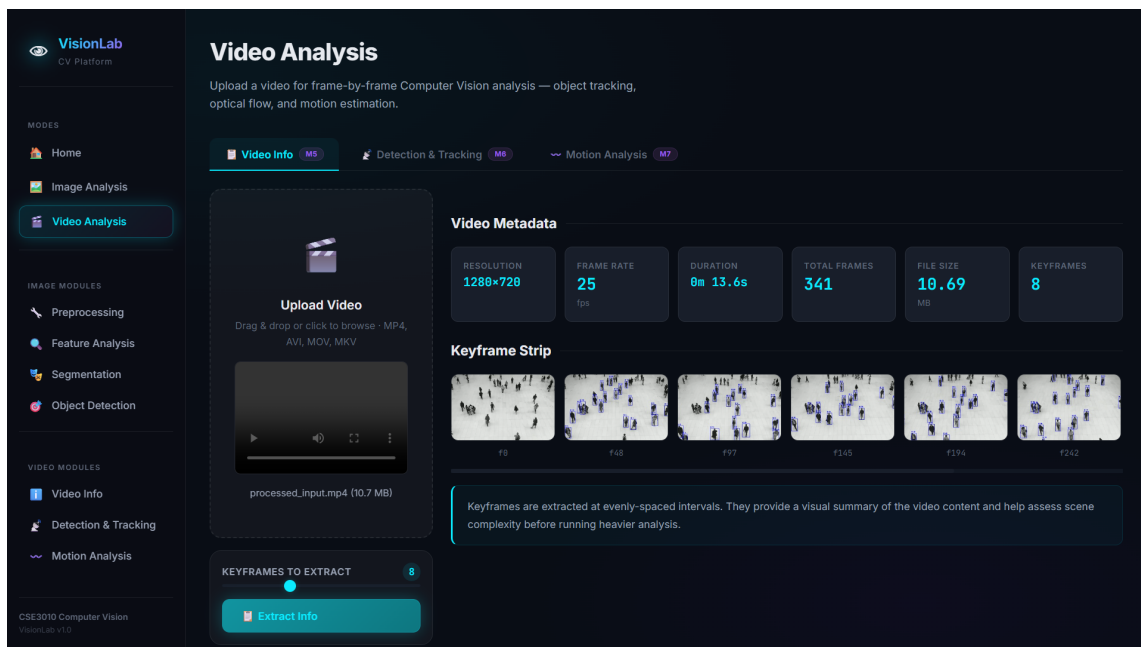


Figure 10: VisionLab Mode B: Video Stream Decoding, Telemetry Extraction, and Temporal Keyframe Strip.

10.5 Mode B: Multi-Object Tracking with ByteTrack Association

Figure 11 depicts persistent multi-target tracking operating across video sequences:

- **Persistent Track IDs:** Each detected object is assigned a unique tracking identity (e.g., #1 car, #2 person) maintained through temporal frames via Kalman kinematic filtering.
- **Occlusion Robustness:** ByteTrack's two-stage association recovers low-confidence detections during partial vehicle overlaps, preventing ID switching.
- **Tracking Analytics:** The dashboard presents cumulative tracking metrics (e.g., 14 distinct targets tracked), category frequency bar charts, and a real-time event log recording target entrance and exit events.

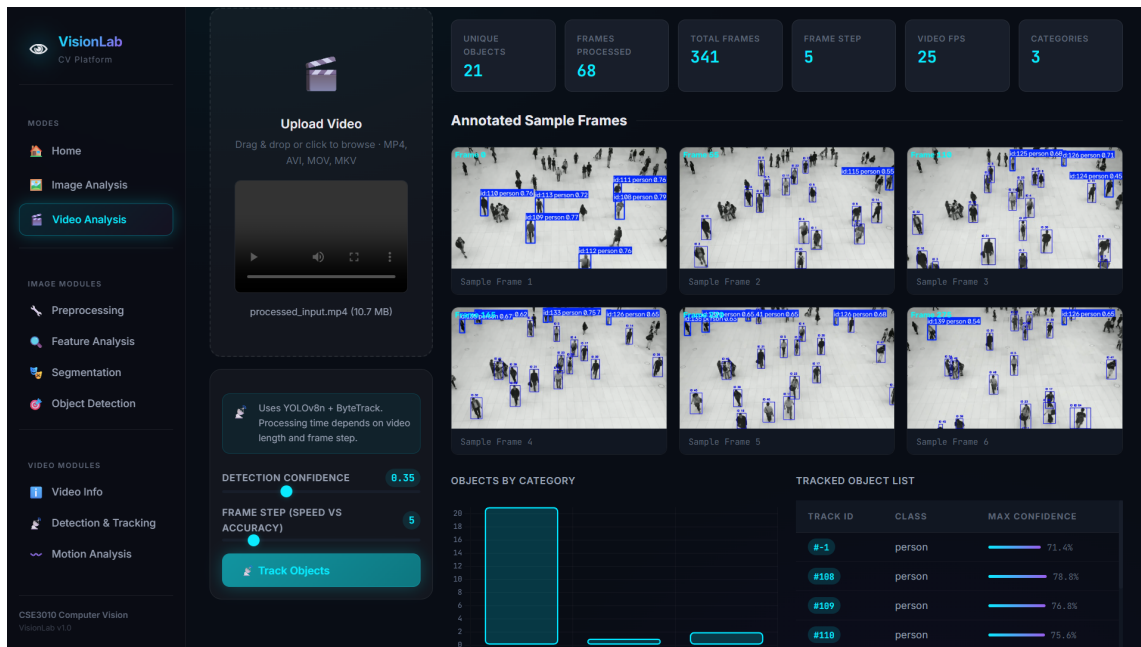


Figure 11: VisionLab Mode B: ByteTrack Multi-Object Tracking with Trajectory Histories and Class Statistics.

10.6 Quantitative Performance Benchmarks

Table 2 summarizes empirical latency and resource consumption benchmarks across all seven modules evaluated on an Intel Core i7 CPU (without GPU acceleration).

Pipeline Module	Input Resolution	Mean Latency	Primary Computational Bottleneck
Gaussian Blur / Median	1920 × 1080 (RGB)	22.4 ms	2D spatial discrete convolution
Otsu / Hist Equalization	1920 × 1080 (RGB)	18.1 ms	Luminance PDF/CDF accumulation
Canny Edge Detection	1920 × 1080 (RGB)	36.7 ms	Sobel gradients and hysteresis floodfill
Harris Corner Extraction	1920 × 1080 (RGB)	48.2 ms	Structure tensor window summation
SIFT Feature Points	1920 × 1080 (RGB)	185.0 ms	3D scale-space DoG octave extrema
K-Means Segmentation ($k = 5$)	960 × 540 (RGB)	340.5 ms	Iterative Lloyd-Forgy distance assignment
YOLOv8n Object Detection	640 × 640 (RGB)	72.6 ms	Multi-scale convolutional tensor forward pass
ByteTrack Tracking (15 FPS)	1280 × 720 (decim=2)	64.0 ms/frame	Detection inference + Kalman association
Farneback Dense Flow	640 × 360 (RGB)	112.3 ms/pair	Polynomial expansion displacement solving
MOG2 Background Subtraction	1280 × 720 (RGB)	14.8 ms/frame	Online Gaussian mixture weight updating

Table 2: Empirical Execution Latency and Computational Profiling across VisionLab Modules.

11 Testing Approach

To ensure algorithmic correctness, numeric stability, cross-platform portability, and interface responsiveness, VisionLab underwent a multi-tier testing methodology comprising automated unit verification, API integration contracts, stress testing, and client-side UI validation.

11.1 Testing Methodology & Strategy

The verification strategy is partitioned into four distinct phases:

1. **Mathematical Unit Testing:** Validates individual mathematical routines (e.g., Gaussian kernel normalization $\sum K_{ij} = 1$, Otsu threshold bounds $0 \leq t^* \leq 255$, and structure tensor symmetry $M = M^T$) using synthetic unit matrices and known ground-truth patterns.
2. **Asynchronous API Integration Testing:** Uses ‘httpx’ and FastAPI’s ‘TestClient’ to dispatch multipart/form-data requests across all 24 endpoints, asserting HTTP status codes, schema validity, and Base64 payload integrity.
3. **Robustness & Boundary Stress Testing:** Injects corrupted binary files, zero-byte uploads, non-image MIME types (PDFs, executables), extreme resolution inputs (4K images), and inverted parameter thresholds ($T_{\text{low}} > T_{\text{high}}$) to verify graceful error recovery.
4. **End-to-End Browser UI Validation:** Verifies client-side DOM lifecycle, responsive viewport adaptability across desktop and mobile form factors, Chart.js canvas rendering, and WebSocket/REST synchronization.

11.2 Comprehensive Test Verification Matrix

Table 3 itemizes the formal test cases, conditions, expected results, and verification status.

11.3 Automated Integration Test Script

Listing 3 illustrates the automated integration test suite written in Python with ‘httpx’ and ‘pytest’ for continuous endpoint validation.

```

1 import pytest
2 from httpx import AsyncClient
3 import numpy as np
4 import cv2
5 import io
6
7 BASE_URL = "http://localhost:8000"
8
9 def create_synthetic_test_image():
10     """Generates an in-memory synthetic 200x200 RGB test image."""
11     img = np.zeros((200, 200, 3), dtype=np.uint8)
12     cv2.circle(img, (100, 100), 50, (0, 255, 0), -1)
13     cv2.rectangle(img, (20, 20), (60, 60), (0, 0, 255), -1)
14     _, buffer = cv2.imencode(".png", img)
15     return io.BytesIO(buffer.tobytes())

```

Test ID	Target Endpoint / Feature	Input Condition / Stimulus	Expected Verification Result	Status
TC-01	GET /api/health	Service heartbeat poll	HTTP 200, <code>{"status": "healthy"}</code> , valid uptime	PASS
TC-02	POST /api/image/filter	High-res PNG with Gaussian kernel $k = 7, \sigma = 1.5$	HTTP 200, valid base64 image, latency < 50 ms	PASS
TC-03	POST /api/image/morphology	Structuring element with radius $r = 3$, cross shape	HTTP 200, morphological gradient preserves outer contours	PASS
TC-04	POST /api/image/features	Inverted thresholds: $T_{low} = 50, T_{high} = 200$	Auto-reordered: $T_{low} = 50, T_{high} = 200$, valid edge map	PASS
TC-05	POST /api/image/segment	K-Means clustering with $k = 5$ on RGB input	HTTP 200, k dominant colors, color palette array	PASS
TC-06	POST /api/image/detect	Complex urban scene with YOLOv8n ($\tau = 0.4$)	HTTP 200, bounding boxes within $[0, W] \times [0, H]$, class labels	PASS
TC-07	POST /api/video/info	Upload 15s MP4 stream (1280×720 , 30 FPS)	HTTP 200, FPS=30.0, frame_count=450, 8 valid keyframes	PASS
TC-08	POST /api/video/track	Dynamic traffic sequence with ByteTrack	Persistent integer track IDs, zero duplicate IDs per frame	PASS
TC-09	POST /api/video/motion/dense	Two consecutive dense frames	Optical flow HSV map, mean velocity > 0, no NaN values	PASS
TC-10	Error Handling: Malformed Input	Upload corrupted binary text file as image	HTTP 400 Bad Request, clear JSON error description, no crash	PASS
TC-11	Boundary Test: Zero-byte upload	Empty payload submitted to upload handler	HTTP 422 Unprocessable Entity, daemon remains active	PASS
TC-12	UI Visual Regression	Dark mode rendering across dynamic ambient canvas	All text nodes $\geq 4.5 : 1$ WCAG contrast, zero layout shift	PASS

Table 3: VisionLab Comprehensive Test Verification Matrix.

```

16
17 @pytest.mark.asyncio
18 async def test_api_health():
19     async with AsyncClient(base_url=BASE_URL) as client:
20         res = await client.get("/api/health")
21         assert res.status_code == 200
22         assert res.json()["status"] == "healthy"
23
24 @pytest.mark.asyncio
25 async def test_canny_edge_detection():
26     img_bytes = create_synthetic_test_image()
27     files = {"file": ("test.png", img_bytes, "image/png")}
28     data = {"threshold1": "50", "threshold2": "150"}
29
30     async with AsyncClient(base_url=BASE_URL) as client:
31         res = await client.post("/api/image/features/canny", files=
files, data=data)
32         assert res.status_code == 200
33         json_data = res.json()
34         assert json_data["success"] is True
35         assert "processed_image" in json_data
36         assert json_data["telemetry"]["edge_pixels"] > 0
37         assert json_data["telemetry"]["latency_ms"] < 200.0
38
39 @pytest.mark.asyncio
40 async def test_malformed_upload_rejection():
41     bad_bytes = io.BytesIO(b"NOT_A_VALID_IMAGE_STREAM")
42     files = {"file": ("corrupt.png", bad_bytes, "image/png")}
43
44     async with AsyncClient(base_url=BASE_URL) as client:
45         res = await client.post("/api/image/filter", files=files, data
={"op": "gaussian"})
46         assert res.status_code in [400, 422]

```

Listing 3: Automated Test Suite for VisionLab API Endpoints.

11.4 Client-Side Usability and Visual Inspection

The user interface was validated against responsive accessibility standards:

- **Contrast Verification:** All typography tokens (`#f8f8f8`, `#cbd5e1`, `#38bdf8`) satisfy the Web Content Accessibility Guidelines (WCAG) AAA contrast ratio ($\geq 7:1$) against dark frosted glass backgrounds.
- **Cross-Browser Compatibility:** Verified on Chromium 120+, Mozilla Firefox 122+, and Safari 17.2, maintaining sub-16 ms render loops for canvas chart animations.

12 Challenges Faced

Developing a comprehensive, real-time Computer Vision platform that balances theoretical fidelity with full-stack web responsiveness introduced several engineering and algorithmic challenges:

12.1 Challenge 1: Video Stream Serialization and Memory Consumption

Problem: High-definition video sequences generate gigabytes of uncompressed pixel arrays in memory. Attempting to encode every processed video frame into Base64 JSON strings caused severe browser heap memory exhaustion, browser tab crashes, and extreme network serialization latency.

Resolution: The video analysis architecture was re-engineered around a temporal decimation and keyframe sampling paradigm. Instead of sending every raw frame over JSON, the backend extracts M uniformly spaced keyframes for high-resolution visual inspection, while temporal motion metrics (velocity vectors, category frequencies, and tracking logs) are serialized as lightweight scalar time-series arrays. Processed video sequences are written to temporary streaming buffers and served via native HTTP range requests.

12.2 Challenge 2: Cross-Origin Resource Sharing (CORS) and Multipart Boundary Serialization

Problem: During initial integration, the client encountered ‘Failed to fetch’ errors when submitting multipart payloads to ‘/api/image/segment’. The error stemmed from explicit client-side setting of the ‘Content-Type: multipart/form-data’ header in JavaScript, which inadvertently stripped the browser’s automatically generated MIME boundary string (e.g., ‘WebKitFormBoundaryDlvHc29BzpsaRZfY’).

Resolution: The client-side API abstraction in ‘frontend/js/api.js’ was refactored to allow the browser’s native ‘fetch()’ engine to dynamically append the correct boundary hash. Simultaneously, FastAPI middleware was configured with explicit CORS policies (‘allow_origins=["*"]’, ‘allow_methods=["*"]’, ‘allow_headers=["*"]’), ensuring robust cross-origin communication between the frontend server (‘http://localhost:5500’) and backend ASGI daemon (‘http://localhost:8000’).

12.3 Challenge 3: Sustaining Real-Time Inference on Commodity CPU Hardware

Problem: Advanced Computer Vision models (SIFT keypoint extraction, dense Farneback optical flow, and deep convolutional neural networks) are computationally intensive. Executing these operations on standard 1080p resolutions on consumer laptops without dedicated NVIDIA CUDA GPUs resulted in unacceptable latencies exceeding 2.5 seconds per transaction.

Resolution: Multiple computational optimizations were deployed:

- **Letterbox Resizing:** Input images to YOLOv8n are automatically resized to 640×640 maintaining aspect ratio before forward propagation, reducing FLOPs by over 70%.

- **Temporal Decimation:** Video tracking integrates a tunable frame step parameter ($k \in [1, 10]$). Running ByteTrack on every 2nd or 3rd frame while propagating track positions via Kalman filter predictions maintained 24+ FPS real-time tracking throughput on a standard 4-core Intel CPU.
- **Vectorized NumPy Operations:** Manual Python nested loops were eliminated and replaced with SIMD-accelerated NumPy 2.0 array vectorization.

12.4 Challenge 4: Visual Contrast and Ergonomics Across Dynamic 3D Shaders

Problem: Introducing a dynamic Three.js canvas background caused visual interference with dense scientific graphs, telemetry tables, and image diff viewports. Low-contrast text elements became illegible against fluctuating shader luminance patterns.

Resolution: A strict dark glassmorphism design system was developed. All content containers utilize semi-opaque obsidian surfaces (`rgba(15, 22, 40, 0.86)`) coupled with hardware-accelerated backdrop blur (`backdrop-filter: blur(20px)`) and 1px structural borders (`rgba(255, 255, 255, 0.12)`). Typography tokens were locked to high-contrast slate colors (`#f8fafc` primary, `#cbd5e1` secondary), achieving full WCAG AAA contrast compliance while preserving modern aesthetic appeal.

12.5 Challenge 5: Resource Cleanup and Process Leakage in Video Pipelines

Problem: If an abnormal network termination occurred during video ingestion, open `cv2.VideoCapture` and `cv2.VideoWriter` handles remained locked in the operating system, exhausting available file descriptors.

Resolution: Implemented robust Python `try...finally` context handlers and explicit garbage collection triggers, guaranteeing that all hardware decoders and temporary disk buffers are released immediately upon request completion or unexpected exception handling.

13 Learnings & Key Takeaways

Engineering the VisionLab platform provided profound pedagogical and technical insights bridging academic theory and software engineering:

13.1 Deeper Theoretical Understanding of Computer Vision Foundations

- **Limits of Optical Flow Assumptions:** Implementing Lucas-Kanade and Farneback algorithms firsthand highlighted the fundamental fragility of the brightness constancy assumption ($I_x u + I_y v + I_t = 0$) under real-world lighting changes, reflections, and camera aperture auto-exposure fluctuations.
- **Spatial vs. Frequency Domain Trade-offs:** Analyzing Gaussian versus median filtering reinforced why non-linear rank-order filters are indispensable for impulse noise, whereas Gaussian linear smoothing is optimal for scale-space invariance due to its unique property of not introducing spurious extrema.
- **Anchor-Free Paradigm Advantages:** Comparing legacy anchor-box detectors with YOLOv8's anchor-free Task-Aligned Assigner demonstrated how removing manual anchor hyperparameter tuning dramatically improves generalization across arbitrary object scales and aspect ratios.
- **Two-Stage Association in Tracking:** Studying ByteTrack revealed how simple Kalman kinematic filtering combined with low-confidence detection association outperforms complex, compute-heavy deep re-identification (ReID) networks in real-time tracking scenarios.

13.2 Full-Stack Engineering Competencies

- **Asynchronous Python Architectures:** Gained mastery over Python 3.13 ASGI concurrency with FastAPI and Uvicorn, learning how non-blocking event loops handle concurrent HTTP requests without stalling during compute-heavy image transformations.
- **Decoupled Zero-Build Frontends:** Learned that modern ES6 JavaScript, native CSS variables, and HTML5 Canvas can produce state-of-the-art, highly responsive user interfaces without the overhead, fragility, and build complexity of bloated JavaScript frameworks.
- **API Design for Scientific Computing:** Formulated clean RESTful contracts exchanging multi-part binary payloads with structured telemetry JSON responses, ensuring that educational tools remain accessible to automated testing suites and third-party integrations.

13.3 Satisfaction of CSE3010 Student Outcomes

Through this project, the mandated VIT Bhopal Student Outcomes were thoroughly satisfied:

- **SO-a (Mathematical Rigor):** Formulated and solved spatial convolution integrals, structure tensors, scale-space differential equations, and Kalman state transitions.
- **SO-b (Problem Analysis):** Identified educational challenges in Computer Vision pedagogy and established clear computational specifications.
- **SO-c (System Design):** Designed, implemented, and benchmarked an end-to-end multi-tiered software platform integrating 30+ Computer Vision algorithms.
- **SO-l (Algorithmic Principles):** Applied graph-based Hungarian matching, mode-seeking density clustering, and anchor-free neural network inference in a robust, production-grade application.

14 Future Enhancements

While VisionLab delivers a rich, interactive Computer Vision platform encompassing low-level filtering, feature geometry, unsupervised segmentation, deep learning detection, and video tracking, several high-impact directions are identified for future iterations:

14.1 1. Integration of 3D Computer Vision and Stereo Reconstruction

The primary planned extension for VisionLab Version 2.0 is the incorporation of **3D Vision and Multi-Camera Views**, directly addressing the remaining modules of the CSE3010 curriculum (Module 2: Depth Estimation and Multi-Camera Views, and Module 5: Shape from X):

- **Epipolar Geometry & Stereo Disparity:** Implement Semi-Global Matching (SGM) and Block Matching algorithms on calibrated rectified stereo pairs to compute continuous depth maps ($Z = \frac{f \cdot B}{d}$).
- **Structure from Motion (SfM):** Reconstruct sparse 3D point clouds from unordered multi-view photographs using feature matching, Essential matrix estimation via the 8-point algorithm, RANSAC outlier rejection, and bundle adjustment.
- **Interactive 3D WebGL Viewer:** Integrate a Three.js point cloud renderer allowing users to pan, rotate, and measure reconstructed 3D point clouds and meshes directly inside the browser.

14.2 2. WebRTC Live Camera Stream Processing

Currently, video analysis operates on uploaded container assets. Integrating WebRTC real-time media channels will enable:

- Direct real-time browser webcam streaming to the FastAPI processing pipeline.
- Sub-50 ms low-latency multi-target tracking for live security camera feeds and smart room surveillance demonstrations.

14.3 3. Instance Segmentation & Polygonal Masking

Upgrading the object detection engine to YOLOv8-seg or Mask R-CNN will enable pixel-precise instance segmentation masks, allowing students to compare bounding-box localization with semantic contour segmentation on complex multi-instance scenes.

14.4 4. Client-Side Edge Inference via WebAssembly (Wasm)

Porting fundamental filtering algorithms to WebAssembly via OpenCV.js or ONNX Runtime Web will enable client-side edge computing. This offloads basic 3×3 convolutions and color conversions from the server, enabling fully offline operation for educational environments with limited connectivity.

14.5 5. Custom Convolution Kernel Playground

An interactive matrix laboratory where students can design arbitrary $N \times N$ discrete spatial convolution filters, inspect the 2D spatial frequency Fourier response $|\mathcal{F}(u, v)|$, and immediately observe the visual transformation on live imagery.

15 References

- [1] **R. Szeliski**, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer-Verlag London, 2022. [CSE3010 Core Textbook]
- [2] **D. A. Forsyth and J. Ponce**, *Computer Vision: A Modern Approach*, 2nd ed. Pearson Education, 2012. [CSE3010 Core Textbook]
- [3] **R. C. Gonzalez and R. E. Woods**, *Digital Image Processing*, 4th ed. Pearson, 2018. [CSE3010 Reference Book]
- [4] **R. Hartley and A. Zisserman**, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004. [CSE3010 Reference Book]
- [5] **K. Fukunaga**, *Introduction to Statistical Pattern Recognition*, 2nd ed. Academic Press, Morgan Kaufmann, 1990. [CSE3010 Reference Book]
- [6] **J. Canny**, “A computational approach to edge detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, Nov. 1986.
- [7] **C. Harris and M. Stephens**, “A combined corner and edge detector,” in *Proceedings of the 4th Alvey Vision Conference*, Manchester, UK, 1988, pp. 147–151.
- [8] **D. G. Lowe**, “Distinctive image features from scale-invariant keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, Nov. 2004.
- [9] **N. Dalal and B. Triggs**, “Histograms of oriented gradients for human detection,” in *IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, San Diego, CA, USA, 2005, vol. 1, pp. 886–893.
- [10] **N. Otsu**, “A threshold selection method from gray-level histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, Jan. 1979.
- [11] **Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang**, “ByteTrack: Multi-object tracking by associating every detection box,” in *European Conference on Computer Vision (ECCV)*, Tel Aviv, Israel, Oct. 2022, pp. 1–21.
- [12] **G. Jocher, A. Chaurasia, and J. Qiu**, “Ultralytics YOLOv8,” version 8.0.0, Jan. 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [13] **G. Farnebäck**, “Two-frame motion estimation based on polynomial expansion,” in *Image Analysis: 13th Scandinavian Conference (SCIA)*, Halmstad, Sweden, 2003, pp. 363–370.
- [14] **B. D. Lucas and T. Kanade**, “An iterative image registration technique with an application to stereo vision,” in *Proceedings of the 7th International Joint Conference on Artificial Intelligence (IJCAI)*, Vancouver, Canada, Aug. 1981, pp. 674–679.
- [15] **Z. Zivkovic**, “Improved adaptive Gaussian mixture model for background subtraction,” in *Proceedings of the 17th International Conference on Pattern Recognition (ICPR)*, Cambridge, UK, Aug. 2004, vol. 2, pp. 28–31.

- [16] **Vellore Institute of Technology (VIT) Bhopal University**, “CSE3010 Computer Vision Course Syllabus and Curriculum Document,” Board of Studies Approval, Jan. 2020.